

Emne 3 ggplot1

Visualisering - ggplot2 dag 1



“Visualisering giver dig svar på spørgsmål, du ikke vidste, du havde.” – Ben Schneiderman

Læringsmålene og videoer

Dette kapitel giver en introduktion til, hvordan man visualiserer data med R-pakken **ggplot2**. Du skal være i stand til at:

- Forstå hvad “Grammar of Graphics” betyder og sammenhængen med **ggplot2**-pakken
- Lær at bruge funktionen **ggplot** og dens relevante **geoms** (`geom_point()`, `geom_bar()`, `geom_histogram()`, `geom_boxplot()`, `geom_density()` `geom_line()`)
- Lav en ‘færdig’ figur med en titel og korrekte etiketter på akserne
- Begynd at arbejde med farver og temaer

Hvad er ggplot2?

De fleste i kurset har brugt funktionen `plot()`, som er den standard base-R funktion til at lave et plot. Man kan fortsætte med at bruge plotfunktioner fra base-pakken, men det kan være meget tidskrævende, især når man vil lave mere komplekse og æstetisk plots.

En alternativ løsning er pakken **ggplot2**, som står for “Grammar of Graphics” (se nærmere forklaring nedenfor). **ggplot2** er den mest populære pakke fra **tidyverse**. Som vi vil se i dette kapitel, har den en ret logisk tilgang, hvor man opbygger et plot vha. forskellige komponenter. Det kan virke uoverskueligt i starten, men når man først har lært det, er det faktisk meget intuitivt. Det nyttige i at lære **ggplot2** kan også ses når man begynder at integrere de andre **tidyverse**-pakker fra kapitel 4.

Brugen af materialerne

Vi starter med ‘quick-start’ RStudio eksempler i videoerne. De er dog ikke designet til at indeholde alle detaljer, men til at fungere som udgangspunkt for at komme i gang med øvelserne. Vær venligst opmærksom på, at al den kode, der vises i videoerne, også findes i kursusnotaterne, hvis du selv vil prøve den. Vi anbefaler, at du bruger kursusnotaterne som en reference gennem kurset, når man arbejder på opgaverne. Vær venligst også opmærksom på, at nogle gange bliver nye aspekter også introduceret i selve øvelserne.

Tjekliste til Kapitel 3: ggplot2

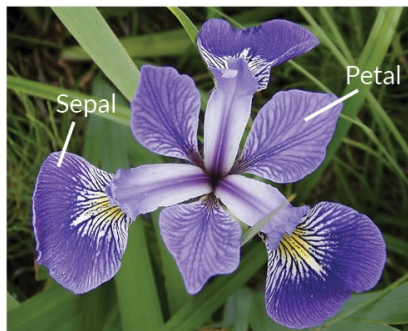
- Se videoerne
- Kig igennem kursusnotaterne
- Lav quiz “ggplot2 - dag 1”
- Lav problemstillingerne

Video ressourcer

- Video 1: Det første plot med ggplot2
- Video 2: Boxplots
- Video 3: Barplots
- Video 4: Histogram og density plots

Transition fra base R til ggplot2

Vi starter som udgangspunkt med base-R og viser, hvordan man laver et lignende plot med **ggplot2**. Til dette formål bruger vi det indbyggede datasæt, der hedder **iris**. Datasættet er meget berømt, og det er næsten sikkert, at du allerede er stødt på det uden for dette kursus, enten på nettet eller i forbindelse med andre kurser, som handler om R. Datasættet var oprindeligt samlet af statistikeren og biologen Ronald Fisher i 1936 og indeholder 50 stikprøver, der dækker forskellige målinger for hver af tre arter af planten iris (Iris setosa, Iris virginica og Iris versicolor).



Iris Versicolor



Iris Setosa



Iris Virginica

Som vi også så i grundlæggende R, kan man indlæse et indbyggede datasæt med hjælp af funktionen `data()`.

```
data(iris)
```

Først vil vi have et overblik over datasættet. Til at få det bruger vi `summary()`, `str()` og `print()`:

```
summary(iris)
```

```
##      Sepal.Length      Sepal.Width      Petal.Length      Petal.Width      Species
##  Min.      :4.300    Min.      :2.000    Min.      :1.000    Min.      :0.100    setosa      :50
##  1st Qu.:5.100    1st Qu.:2.800    1st Qu.:1.600    1st Qu.:0.300    versicolor:50
##  Median :5.800    Median :3.000    Median :4.350    Median :1.300    virginica  :50
##  Mean   :5.843    Mean   :3.057    Mean   :3.758    Mean   :1.199
##  3rd Qu.:6.400    3rd Qu.:3.300    3rd Qu.:5.100    3rd Qu.:1.800
##  Max.    :7.900    Max.    :4.400    Max.    :6.900    Max.    :2.500
```

```
str(iris)
```

```
## 'data.frame':    150 obs. of  5 variables:
##  $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
##  $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
##  $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
##  $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
##  $ Species      : Factor w/ 3 levels "setosa","versicolor",...: 1 1 1 1 1 1 1 1 1 1 ...
```

```
print(iris)
```

```
##      Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1           5.1           3.5           1.4           0.2    setosa
## 2           4.9           3.0           1.4           0.2    setosa
## 3           4.7           3.2           1.3           0.2    setosa
## 4           4.6           3.1           1.5           0.2    setosa
## 5           5.0           3.6           1.4           0.2    setosa
## 6           5.4           3.9           1.7           0.4    setosa
## 7           4.6           3.4           1.4           0.3    setosa
## 8           5.0           3.4           1.5           0.2    setosa
## 9           4.4           2.9           1.4           0.2    setosa
## 10          4.9           3.1           1.5           0.1    setosa
## 11          5.4           3.7           1.5           0.2    setosa
## 12          4.8           3.4           1.6           0.2    setosa
## 13          4.8           3.0           1.4           0.1    setosa
## 14          4.3           3.0           1.1           0.1    setosa
## 15          5.8           4.0           1.2           0.2    setosa
## 16          5.7           4.4           1.5           0.4    setosa
## 17          5.4           3.9           1.3           0.4    setosa
## 18          5.1           3.5           1.4           0.3    setosa
## 19          5.7           3.8           1.7           0.3    setosa
## 20          5.1           3.8           1.5           0.3    setosa
## 21          5.4           3.4           1.7           0.2    setosa
## 22          5.1           3.7           1.5           0.4    setosa
## 23          4.6           3.6           1.0           0.2    setosa
## 24          5.1           3.3           1.7           0.5    setosa
## 25          4.8           3.4           1.9           0.2    setosa
## 26          5.0           3.0           1.6           0.2    setosa
## 27          5.0           3.4           1.6           0.4    setosa
## 28          5.2           3.5           1.5           0.2    setosa
## 29          5.2           3.4           1.4           0.2    setosa
## 30          4.7           3.2           1.6           0.2    setosa
```

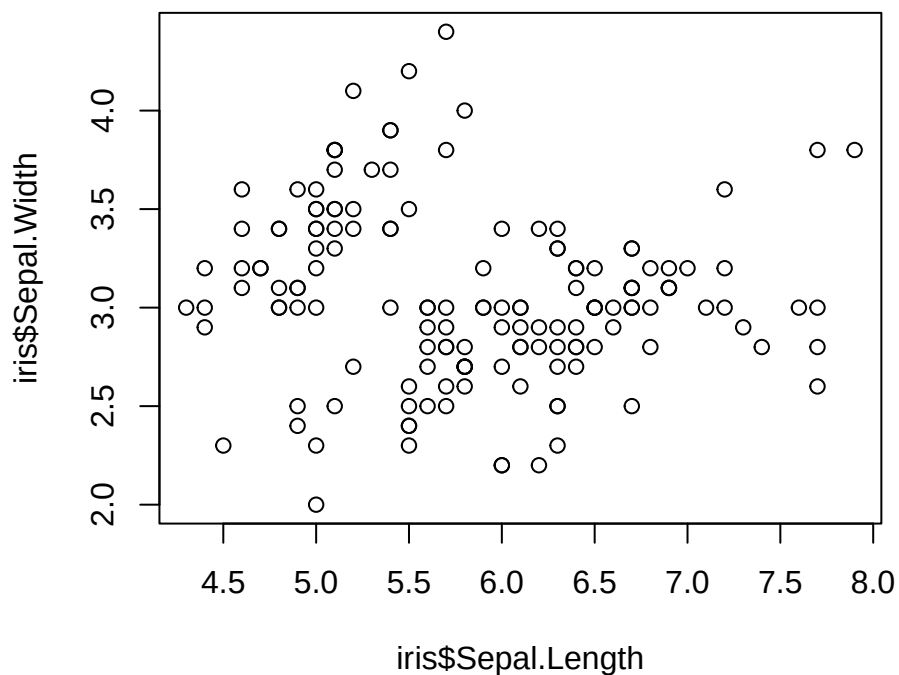
## 31	4.8	3.1	1.6	0.2	setosa
## 32	5.4	3.4	1.5	0.4	setosa
## 33	5.2	4.1	1.5	0.1	setosa
## 34	5.5	4.2	1.4	0.2	setosa
## 35	4.9	3.1	1.5	0.2	setosa
## 36	5.0	3.2	1.2	0.2	setosa
## 37	5.5	3.5	1.3	0.2	setosa
## 38	4.9	3.6	1.4	0.1	setosa
## 39	4.4	3.0	1.3	0.2	setosa
## 40	5.1	3.4	1.5	0.2	setosa
## 41	5.0	3.5	1.3	0.3	setosa
## 42	4.5	2.3	1.3	0.3	setosa
## 43	4.4	3.2	1.3	0.2	setosa
## 44	5.0	3.5	1.6	0.6	setosa
## 45	5.1	3.8	1.9	0.4	setosa
## 46	4.8	3.0	1.4	0.3	setosa
## 47	5.1	3.8	1.6	0.2	setosa
## 48	4.6	3.2	1.4	0.2	setosa
## 49	5.3	3.7	1.5	0.2	setosa
## 50	5.0	3.3	1.4	0.2	setosa
## 51	7.0	3.2	4.7	1.4	versicolor
## 52	6.4	3.2	4.5	1.5	versicolor
## 53	6.9	3.1	4.9	1.5	versicolor
## 54	5.5	2.3	4.0	1.3	versicolor
## 55	6.5	2.8	4.6	1.5	versicolor
## 56	5.7	2.8	4.5	1.3	versicolor
## 57	6.3	3.3	4.7	1.6	versicolor
## 58	4.9	2.4	3.3	1.0	versicolor
## 59	6.6	2.9	4.6	1.3	versicolor
## 60	5.2	2.7	3.9	1.4	versicolor
## 61	5.0	2.0	3.5	1.0	versicolor
## 62	5.9	3.0	4.2	1.5	versicolor
## 63	6.0	2.2	4.0	1.0	versicolor
## 64	6.1	2.9	4.7	1.4	versicolor
## 65	5.6	2.9	3.6	1.3	versicolor
## 66	6.7	3.1	4.4	1.4	versicolor
## 67	5.6	3.0	4.5	1.5	versicolor
## 68	5.8	2.7	4.1	1.0	versicolor
## 69	6.2	2.2	4.5	1.5	versicolor
## 70	5.6	2.5	3.9	1.1	versicolor
## 71	5.9	3.2	4.8	1.8	versicolor
## 72	6.1	2.8	4.0	1.3	versicolor
## 73	6.3	2.5	4.9	1.5	versicolor
## 74	6.1	2.8	4.7	1.2	versicolor
## 75	6.4	2.9	4.3	1.3	versicolor
## 76	6.6	3.0	4.4	1.4	versicolor
## 77	6.8	2.8	4.8	1.4	versicolor
## 78	6.7	3.0	5.0	1.7	versicolor
## 79	6.0	2.9	4.5	1.5	versicolor
## 80	5.7	2.6	3.5	1.0	versicolor
## 81	5.5	2.4	3.8	1.1	versicolor
## 82	5.5	2.4	3.7	1.0	versicolor
## 83	5.8	2.7	3.9	1.2	versicolor
## 84	6.0	2.7	5.1	1.6	versicolor

## 85	5.4	3.0	4.5	1.5 versicolor
## 86	6.0	3.4	4.5	1.6 versicolor
## 87	6.7	3.1	4.7	1.5 versicolor
## 88	6.3	2.3	4.4	1.3 versicolor
## 89	5.6	3.0	4.1	1.3 versicolor
## 90	5.5	2.5	4.0	1.3 versicolor
## 91	5.5	2.6	4.4	1.2 versicolor
## 92	6.1	3.0	4.6	1.4 versicolor
## 93	5.8	2.6	4.0	1.2 versicolor
## 94	5.0	2.3	3.3	1.0 versicolor
## 95	5.6	2.7	4.2	1.3 versicolor
## 96	5.7	3.0	4.2	1.2 versicolor
## 97	5.7	2.9	4.2	1.3 versicolor
## 98	6.2	2.9	4.3	1.3 versicolor
## 99	5.1	2.5	3.0	1.1 versicolor
## 100	5.7	2.8	4.1	1.3 versicolor
## 101	6.3	3.3	6.0	2.5 virginica
## 102	5.8	2.7	5.1	1.9 virginica
## 103	7.1	3.0	5.9	2.1 virginica
## 104	6.3	2.9	5.6	1.8 virginica
## 105	6.5	3.0	5.8	2.2 virginica
## 106	7.6	3.0	6.6	2.1 virginica
## 107	4.9	2.5	4.5	1.7 virginica
## 108	7.3	2.9	6.3	1.8 virginica
## 109	6.7	2.5	5.8	1.8 virginica
## 110	7.2	3.6	6.1	2.5 virginica
## 111	6.5	3.2	5.1	2.0 virginica
## 112	6.4	2.7	5.3	1.9 virginica
## 113	6.8	3.0	5.5	2.1 virginica
## 114	5.7	2.5	5.0	2.0 virginica
## 115	5.8	2.8	5.1	2.4 virginica
## 116	6.4	3.2	5.3	2.3 virginica
## 117	6.5	3.0	5.5	1.8 virginica
## 118	7.7	3.8	6.7	2.2 virginica
## 119	7.7	2.6	6.9	2.3 virginica
## 120	6.0	2.2	5.0	1.5 virginica
## 121	6.9	3.2	5.7	2.3 virginica
## 122	5.6	2.8	4.9	2.0 virginica
## 123	7.7	2.8	6.7	2.0 virginica
## 124	6.3	2.7	4.9	1.8 virginica
## 125	6.7	3.3	5.7	2.1 virginica
## 126	7.2	3.2	6.0	1.8 virginica
## 127	6.2	2.8	4.8	1.8 virginica
## 128	6.1	3.0	4.9	1.8 virginica
## 129	6.4	2.8	5.6	2.1 virginica
## 130	7.2	3.0	5.8	1.6 virginica
## 131	7.4	2.8	6.1	1.9 virginica
## 132	7.9	3.8	6.4	2.0 virginica
## 133	6.4	2.8	5.6	2.2 virginica
## 134	6.3	2.8	5.1	1.5 virginica
## 135	6.1	2.6	5.6	1.4 virginica
## 136	7.7	3.0	6.1	2.3 virginica
## 137	6.3	3.4	5.6	2.4 virginica
## 138	6.4	3.1	5.5	1.8 virginica

```
## 139      6.0      3.0      4.8      1.8 virginica
## 140      6.9      3.1      5.4      2.1 virginica
## 141      6.7      3.1      5.6      2.4 virginica
## 142      6.9      3.1      5.1      2.3 virginica
## 143      5.8      2.7      5.1      1.9 virginica
## 144      6.8      3.2      5.9      2.3 virginica
## 145      6.7      3.3      5.7      2.5 virginica
## 146      6.7      3.0      5.2      2.3 virginica
## 147      6.3      2.5      5.0      1.9 virginica
## 148      6.5      3.0      5.2      2.0 virginica
## 149      6.2      3.4      5.4      2.3 virginica
## 150      5.9      3.0      5.1      1.8 virginica
```

Forestil dig, at vi gerne vil lave et plot, som viser sammenhængen mellem længden og bredden af 'sepals' (bægerblade) - specifikt er vi interesserede i kolonnerne `iris$Sepal.Length` og `iris$Sepal.Width`. Lad os starte med at visualisere variablerne i base-R, ved at bruge `plot`:

```
plot(iris$Sepal.Length, iris$Sepal.Width)
```



Man kan gøre det meget pænere eksempelvis ved at bruge forskellige farver til at betegne de forskellige arter, eller ved at give en hensigtsmæssig overskrift eller aksnavne.

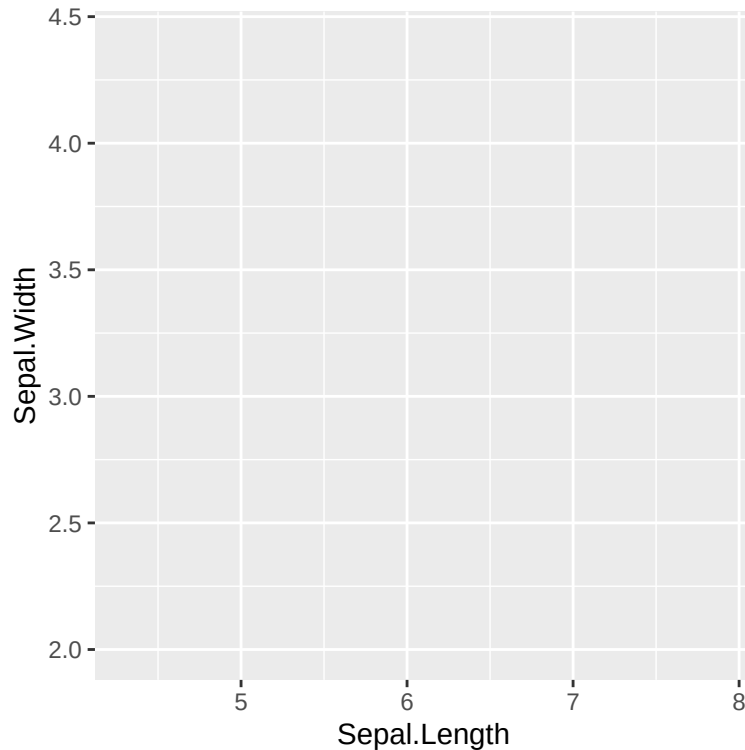
Vores første ggplot

Vi vil imidlertid fokusere på at lave et lignende plot med pakken `ggplot2`. Hvis man ikke allerede har gjort det, så husk at indlæse pakken i R for at få nedenstående koder til at virke.

```
#install.packages("ggplot2") #hvis ikke allerede installeret  
library(ggplot2)
```

For at lave et plot med **ggplot2** tager man altid udgangspunkt i funktionen **ggplot()**. Først specificerer vi vores data - altså at vi vil gerne bruge dataframe **iris**. Dernæst angiver vi indenfor funktionen **aes()** (som sidder indenfor **ggplot()**), at x-aksen skal være **Sepal.Length** og y-aksen **Sepal.Width**. Det ser sådan ud:

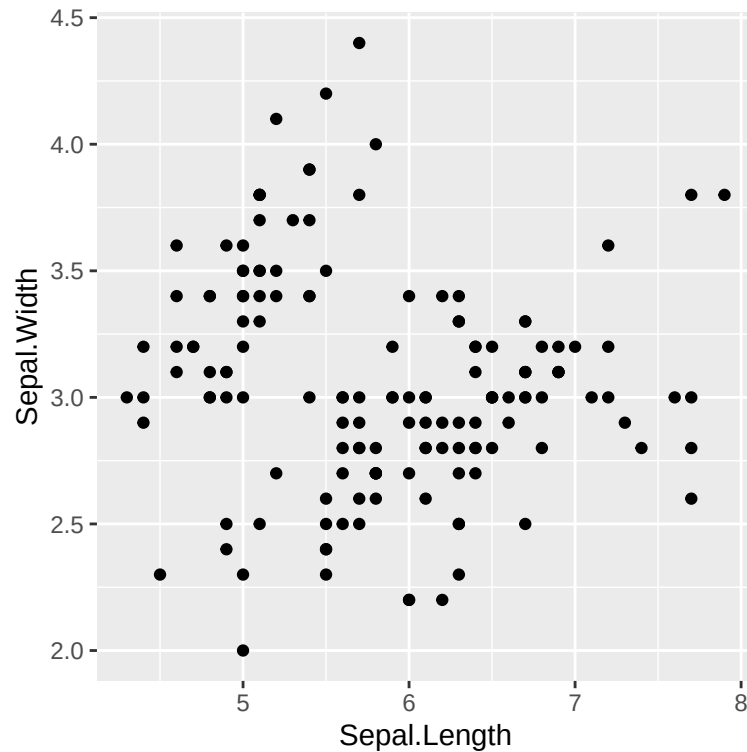
```
ggplot(iris, aes(x=Sepal.Length, y=Sepal.Width))
```



Koden fungerer, men bemærk, at plottet er helt tomt og derfor ikke særligt brugbart. Men der er skabt et grundlag (se aksensavne osv.). Det er tomt fordi vi endnu ikke har specificeret, hvilken plottype det skal være - for eksempel søljediagram/barplot, histogram, punktplot/scatter plot (jeg vælger de engelske begreber herfra for at skabe den bedste sammenhæng med koden). Vi vil gerne bruge et scatter plot, som i **ggplot2** er angivet med funktionen **geom_point()**. Vi tilslutter derfor funktionen **geom_point()** til den **ggplot()** funktion, som vi allerede har specificeret. Husk altid, at man bruger **+** til at forbinde de to “komponenter” (altså **ggplot()** og **geom_point()**) af plottet (ellers får vi fortsat et tomt plot).

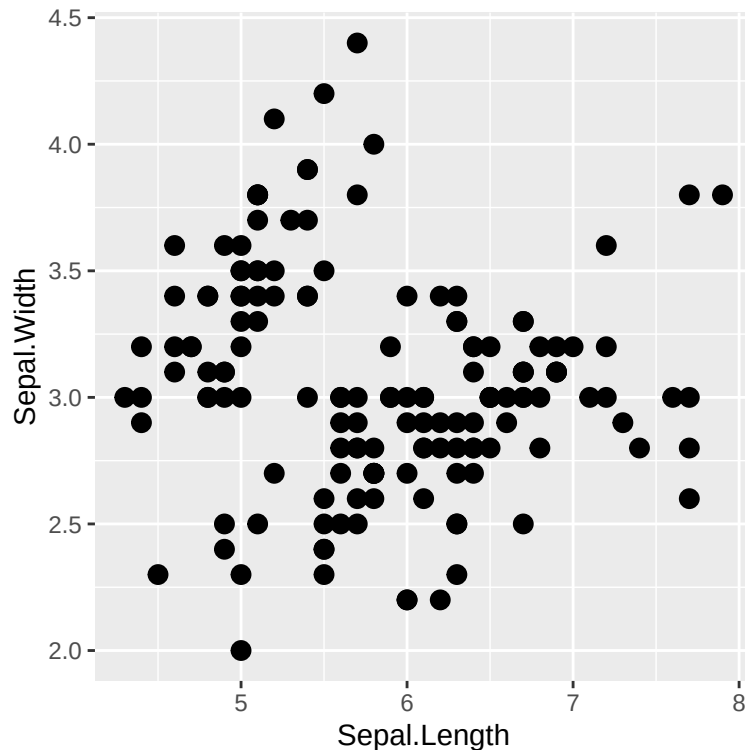
Koden er således:

```
ggplot(iris, aes(x=Sepal.Length, y=Sepal.Width)) +  
  geom_point()
```



Bemærk, at vi ikke har skrevet noget indeni de runde parenteser i funktionen `geom_point()`. Det betyder, at vi accepterer alle standard- eller 'default' parametre, som funktionen tager. Hvis vi vil have noget andet end de standard parametre, kan vi godt specificere det. For eksempel kan vi gøre punkterne lidt større end ved standarden (prøve at tjekke `?geom_point()` for at se en list overfor de mulige parametre, som man kan justere):

```
ggplot(iris, aes(x=Sepal.Length, y=Sepal.Width)) +  
  geom_point(size=3)
```

Vi har nu et plot, som vi kan sammenligne med det ovenstående plot lavet i base-pakken. Ligesom i base-pakken vil vi gerne tilføje nogle ting for at gøre vores plot til vores *færdige figur*. Her i **ggplot2** gøres det ved at tilføje flere komponenter ovenpå, med brugen af `+`, ligesom vi gjorde da vi tilføjede `geom_point()` til `ggplot()`. Men først nogle generelle ord om **ggplot2** og filosofien bag pakken.

Lidt om ggplot2

Syntax

Som vi har lige set, tager `ggplot()` altid udgangspunkt i en dataframe, som vi specificerer først. I `ggplot()` indeholder den dataframe variableerne vi skal bruge til at få lavet figuren. For at gøre det til noget mere konkret: lad os sammenligne koden mellem base-pakken og `ggplot()` for vores iris data. I base-R angav vi direkte vektorer `iris$Sepal.Length` og `iris$Sepal.Width` som parametre `x` og `y`, der tager henholdsvis først og anden-plads i funktionen `plot()`. I `ggplot()` specificerer man istedet først hele datarammen på den første plads, og så bagefter med brugen af `aes()` angiver man hvordan x-aksen og y-aksen ser ud.

```
#baseplot solution
plot(iris$Sepal.Length, iris$Sepal.Width)

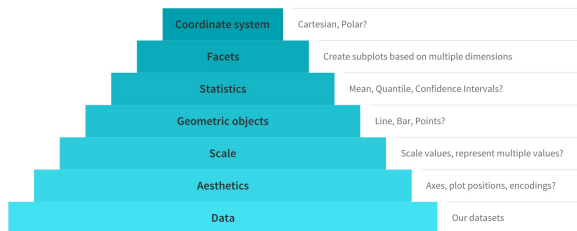
#ggplot2 solution
ggplot(iris, aes(x=Sepal.Length, y=Sepal.Width)) +
  geom_point()
```

En anden fordel af `ggplot2()` er, at man kan fortsætte med at forbedre plottet ved at tilføje ting ovenpå det eksisterende plot, hvilket kan beskrives som en lagbaseret tilgang. Dette gøres intuitivt ved at bruge `+`. Man kan derfor starte med noget simpelt, og gradvist opbygge det noget mere kompleks. Dette er uafhængigt af hvilken type plot, man laver.

Hvad betyder egentlig grammar of graphics?

Den **gg** i **ggplot2** står for *grammar of graphics*, og filosofien er, at man skal definere en *sætningsstruktur* til de figurer, man laver. Med andre ord består vores figur af forskellige komponenter, som man forbinder med “+”.

Major Components of the Grammar of Graphics



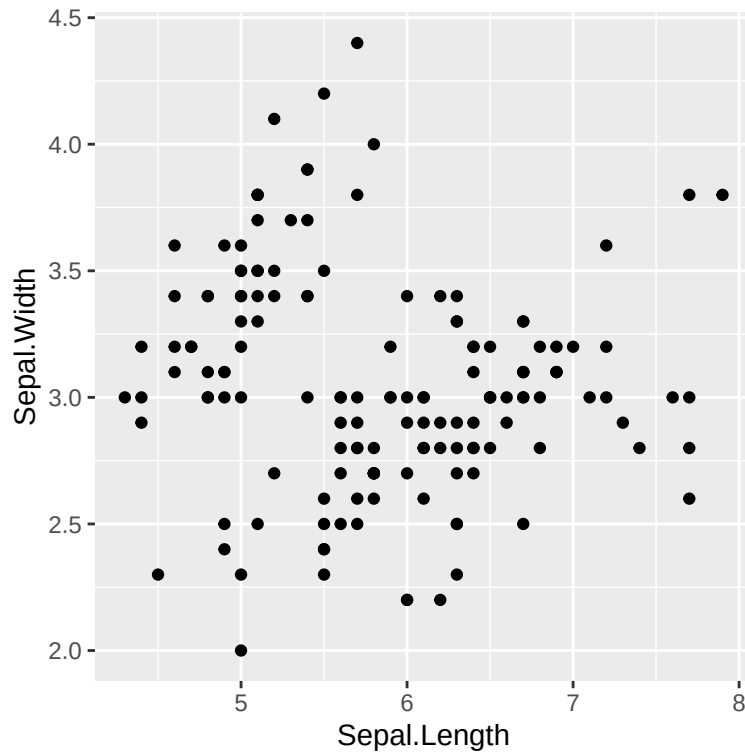
Her er en beskrivelse af de forskellige komponenter, som bruges til at opbygge et plot:

- Data: Datarammer tages altid som udgangspunkt.
- Aesthetics: Variabler til x-aksen eller y-aksen, farve, form eller størrelse
- Scale: Skalering af værdier eller repræsentation af flere værdier
- Geometries: Også kaldet geoms - bestemmer hvilken type plot, der skal laves, som f.eks. søjler, punkter, linjer osv.
- Statistics: Tilføjer f.eks. mean, median eller kvartiler, som beskriver dataene.
- Facets: Opretter subplots baseret på flere dimensioner.
- Coordinate system: Transformerer akser og ændrer afstanden for de viste data.

Global versus lokal æstetik

I de fleste tilfælde bruger man funktionen **aes()** indenfor **ggplot()**, hvilket betyder, at variablerne, der er specificeret inden for **aes()**, gælder globalt over alle komponenter i plottet. Man kan faktisk også skrive en lokal **aes()** inden for selve **geom**-funktionen, som i følgende eksempel:

```
ggplot(iris) +  
  geom_point(aes(x=Sepal.Length, y=Sepal.Width))
```

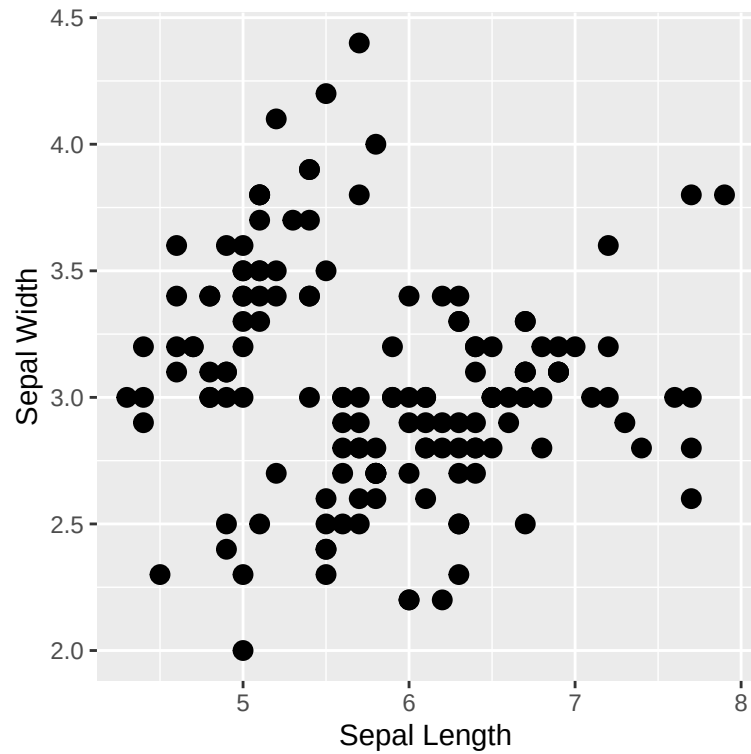


Vi får det samme plot som før, men det er kun `geom_point()`, der er påvirket af specificeringen inden for `aes()`. I simple situationer som dette er der ingen forskel, men når man har mange forskellige komponenter i spil, kan det nogle gange give mening at bruge lokal æstetik.

Specificere etiketter og titel

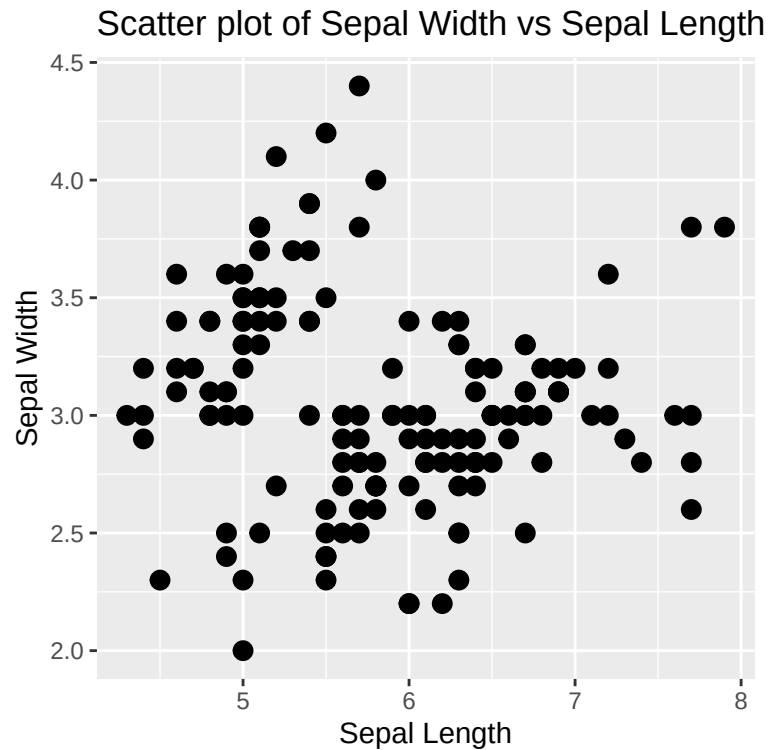
Vi tager udgangspunkt i plottet, som vi har lavet i ovenstående, og prøver at gøre det bedre ved at tilføje nye etiketter og en titel. I `ggplot` kan man opdatere y-akse og x-akse etiketter ved at bruge henholdsvis `ylab` og `xlab`:

```
ggplot(iris, aes(x=Sepal.Length, y=Sepal.Width)) +  
  geom_point(size=3) +  
  xlab("Sepal Length") +  
  ylab("Sepal Width")
```



Vi tilføjer en titel med funktionen `ggtitle()`:

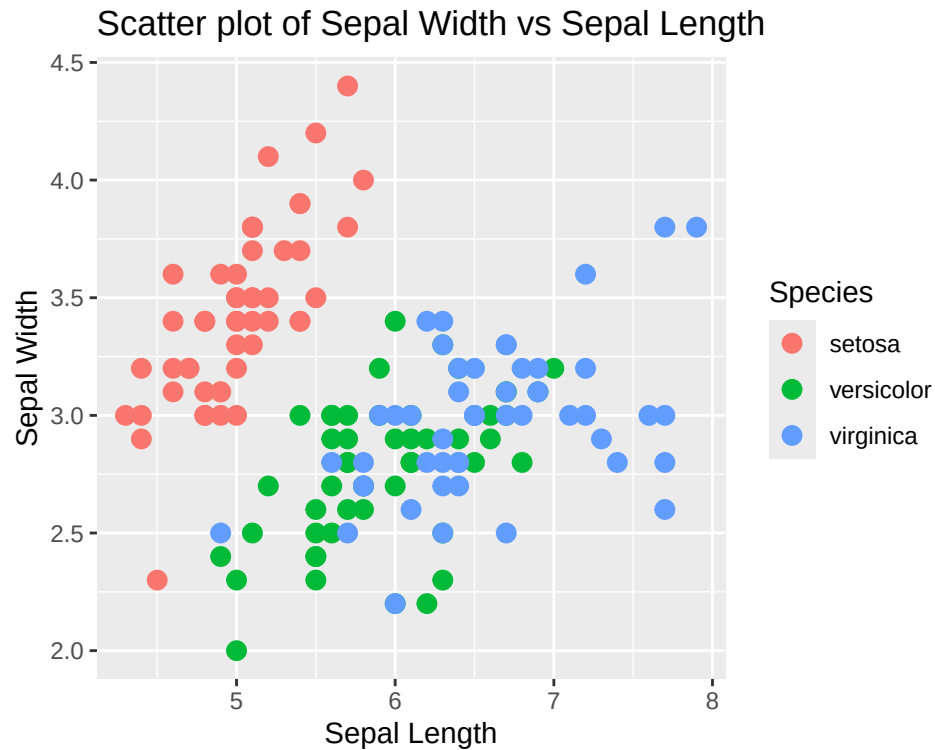
```
ggplot(iris, aes(x=Sepal.Length, y=Sepal.Width)) +  
  geom_point(size=3) +  
  xlab("Sepal Length") +  
  ylab("Sepal Width") +  
  ggtitle("Scatter plot of Sepal Width vs Sepal Length")
```



Ændre farver

I `ggplot2` kan man bruge “automatiske” farver for at skelne mellem de tre forskellige `Species` i datasættet `iris`. I den næste lektion vil vi dække, hvordan man kan være mere fleksibel ved at sætte farver manuelt, men ofte vil vi bare bruge den nemme løsning som udgangspunkt og eventuelt rette op på det senere med en ny komponent, hvis der er behov for det. Vi skriver `color=Species` indenfor `aes()`, som i følgende eksempel. Bemærk, at der kommer en ‘legend’ med, der fortæller os, hvilken art, der får hvilken farve.

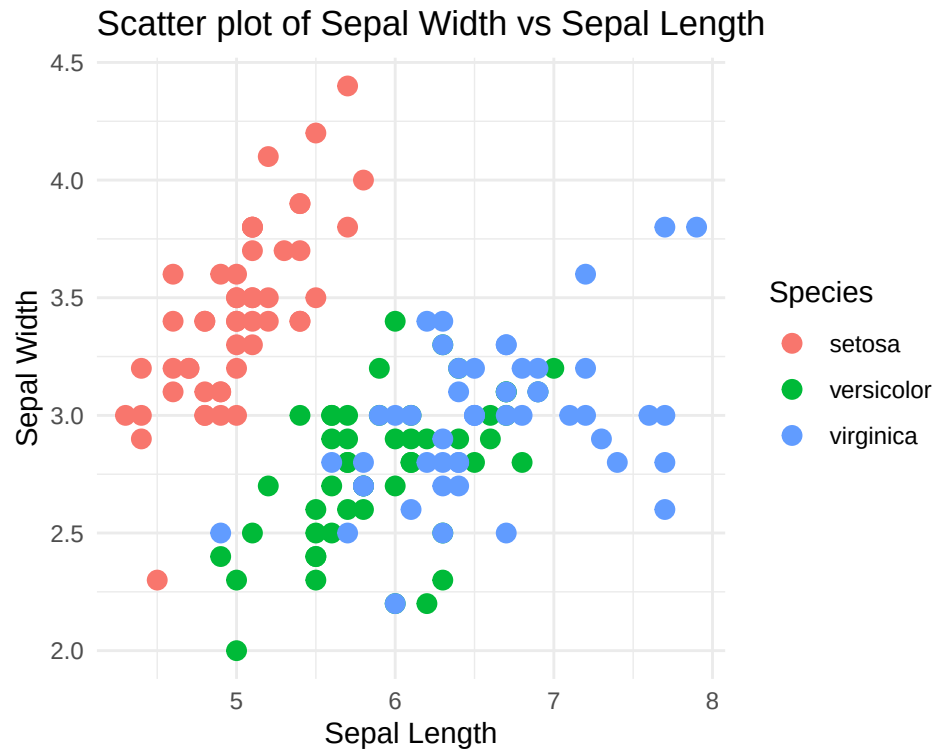
```
ggplot(iris, aes(x=Sepal.Length, y=Sepal.Width, color=Species)) +  
  geom_point(size=3) +  
  xlab("Sepal Length") +  
  ylab("Sepal Width") +  
  ggtitle("Scatter plot of Sepal Width vs Sepal Length")
```



Ændre tema

Det standard tema har en grå baggrund og “grid” linjer, men man kan godt vælge noget andet. For eksempel kan man tilføje `theme_minimal()` som i nedenstående eksempel. Her får vi en hvid baggrund i stedet, mens man stadig får grid linjer. Man kan afprøve forskellige temaer (for eksempel `theme_classic()`, `theme_bw()`), og se, hvilket tema, der fungerer bedst i det enkelte plot.

```
ggplot(iris, aes(x=Sepal.Length, y=Sepal.Width, color=Species)) +  
  geom_point(size=3) +  
  xlab("Sepal Length") +  
  ylab("Sepal Width") +  
  ggtitle("Scatter plot of Sepal Width vs Sepal Length") +  
  theme_minimal()
```



Her er nogle eksempler på mulige temaer, som du kan bruge i dine plotter (det er dog generelt op til dig).

tema
<code>theme_grey()</code>
<code>theme_classic()</code>
<code>theme_bw()</code>
<code>theme_dark()</code>
<code>theme_minimal()</code>
<code>theme_light()</code>
<code>theme_clean()</code>

Se også her, hvis du er interesseret i flere temaer: <https://r-charts.com/ggplot2/themes/>

Forskellige geoms

Indtil videre har vi kun arbejdet med `geom_point()` for at lave et scatter plot, men der er også andre “geoms”, som kan bruges til forskellige typer af plots. Her er en liste over nogle af de mest almindelige geoms:

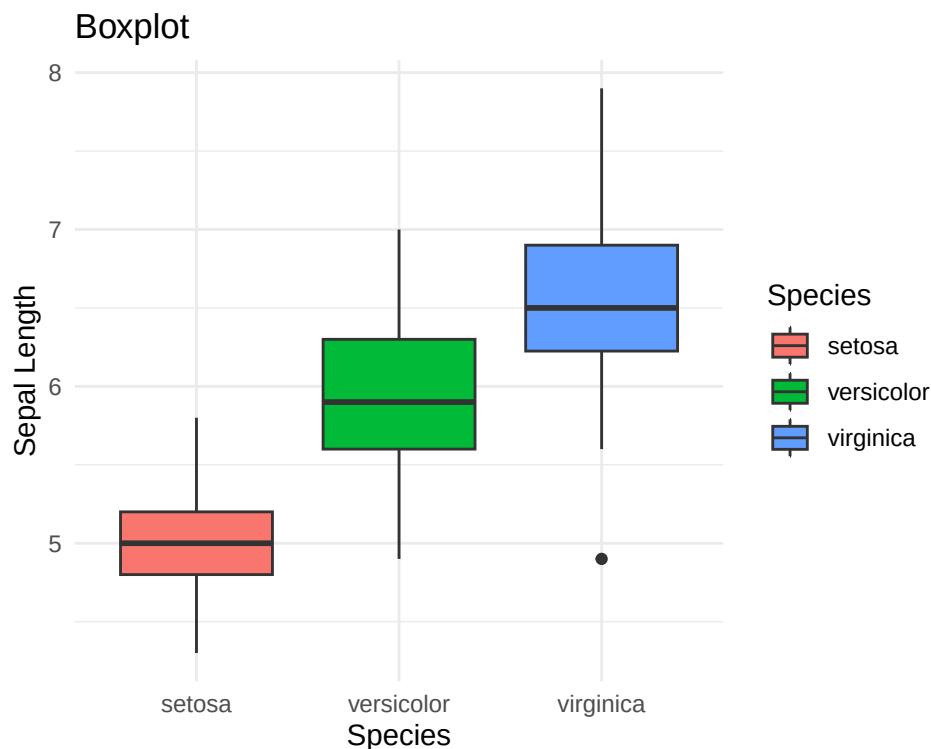
geom	plot
<code>geom_point()</code>	scatter plot
<code>geom_bar()</code>	barplot
<code>geom_boxplot()</code>	boxplot
<code>geom_histogram()</code>	histogram
<code>geom_density()</code>	density

For at lave disse geoms, skal man tilføje dem til `ggplot()`-kommandoen ved at bruge `+`, på samme måde som vi gjorde med `geom_point()`. Der kan dog være specifikke overvejelser, der er værd at have i tankerne for nogle plot-typer, før man bruger dem.

Boxplot (`geom_box`)

For at lave et boxplot af `Sepal.Length` opdelt efter `Species`, angiver vi `Species` på x-aksen og `Sepal.Length` på y-aksen. Vi vil også have, at hver art få sin egen farve, så bruger vi `fill=Species`.

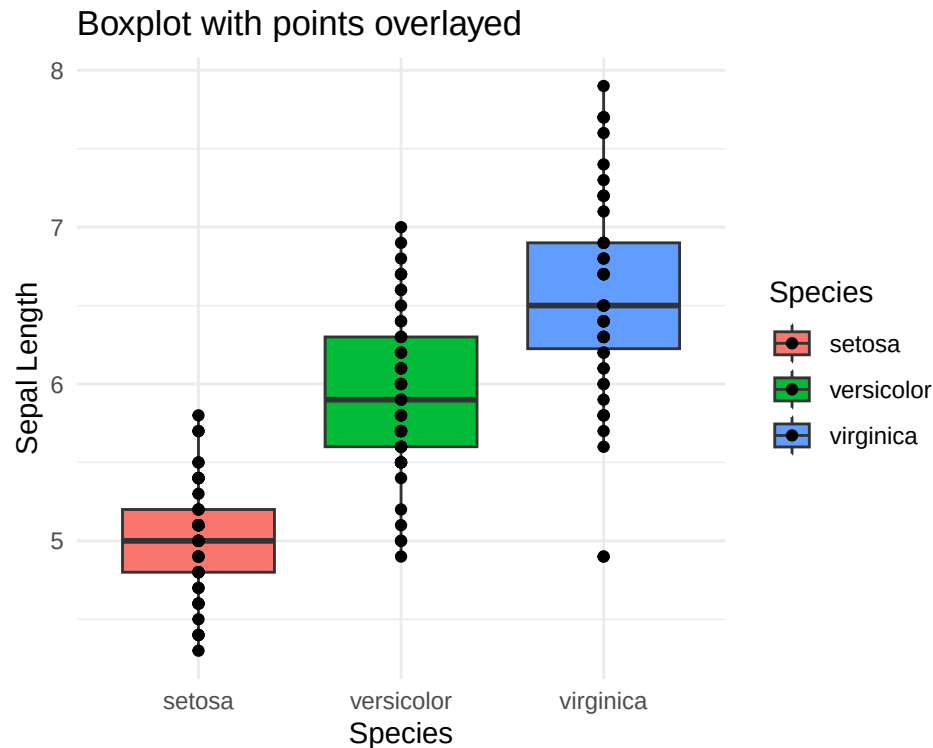
```
ggplot(data=iris, aes(x=Species, y=Sepal.Length, fill=Species)) +  
  geom_boxplot() +  
  ylab("Sepal Length") +  
  ggtitle("Boxplot") +  
  theme_minimal()
```



Lave punkter ovenpå

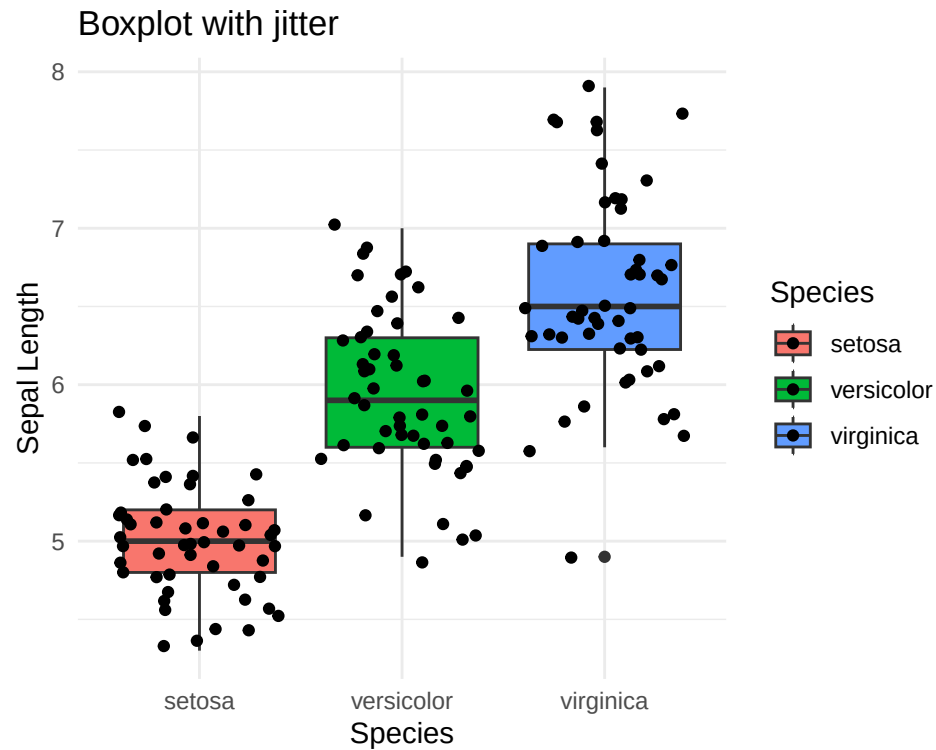
Det kan ofte være nyttigt at plote de faktiske datapunkter oven på et boxplot, så man kan se både fordelingen i dataene samt de rå data. En løsning er at benytte `geom_point()` ved at tilføje det som komponent over vores eksisterende kode.

```
ggplot(data=iris, aes(x=Species, y=Sepal.Length, fill=Species)) +  
  geom_boxplot() +  
  geom_point() +  
  ylab("Sepal Length") +  
  ggtitle("Boxplot with points overlayed") +  
  theme_minimal()
```

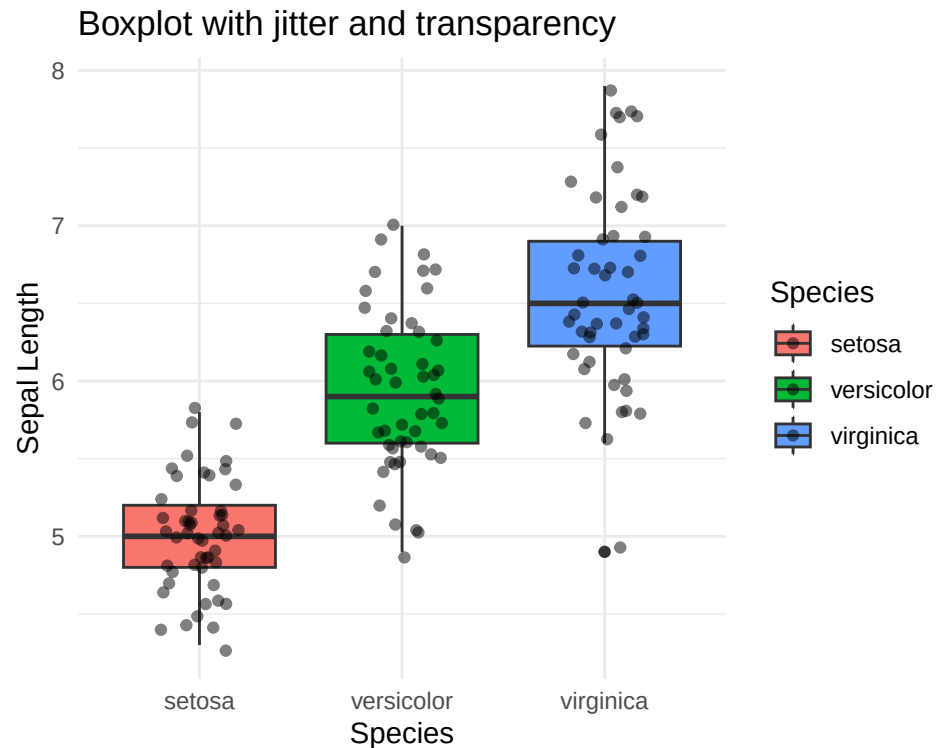
Man kan dog se, at det ikke er særlig informativt, da alle punkter er på den samme lodrette linje. Hvis der er mange punkter med samme eller næsten samme værdier, kan man ikke se de fleste af dem i plottet. En bedre løsning er at indføre noget tilfældighed i punkterne langs x-aksen, så at man tydeligere kan se dem. Dette kaldes “jitter”, og man specificere jitter ved at bruge `geom_jitter()` i stedet for `geom_point()`.

```
ggplot(data=iris, aes(x=Species, y=Sepal.Length, fill=Species)) +  
  geom_boxplot() +  
  geom_jitter() +  
  ylab("Sepal Length") +  
  ggtitle("Boxplot with jitter") +  
  theme_minimal()
```



Man kan også specificere `alpha`, som gøre punkterne mere gennemsigtige og mindre markante. Man kan også ændre på `width`, som kontrollerer deres spredning langs x-aksen.

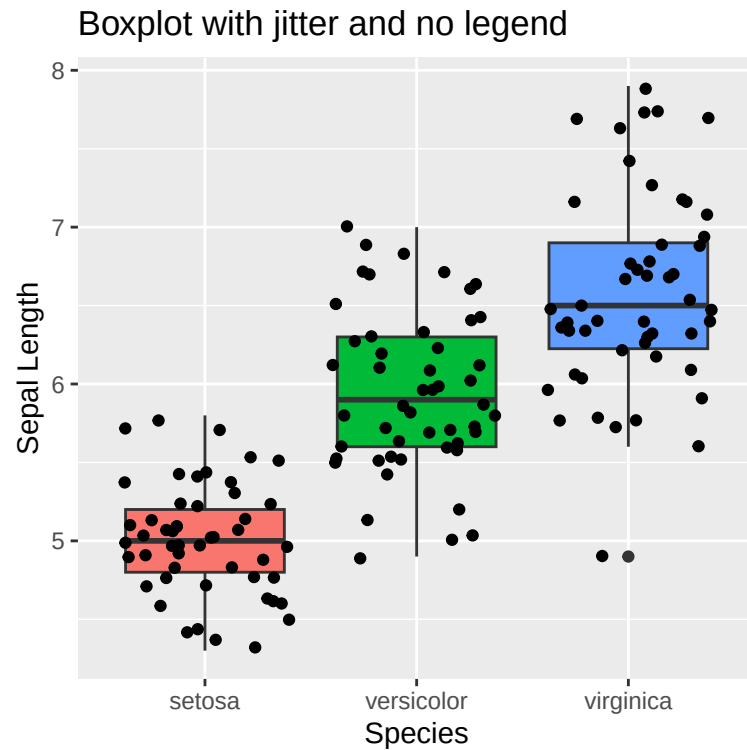
```
ggplot(data=iris, aes(x=Species, y=Sepal.Length, fill=Species)) +  
  geom_boxplot() +  
  geom_jitter(alpha=0.5, width=0.2) +  
  ylab("Sepal Length") +  
  ggtitle("Boxplot with jitter and transparency") +  
  theme_minimal()
```



Fjerne legend hvis unødvendige

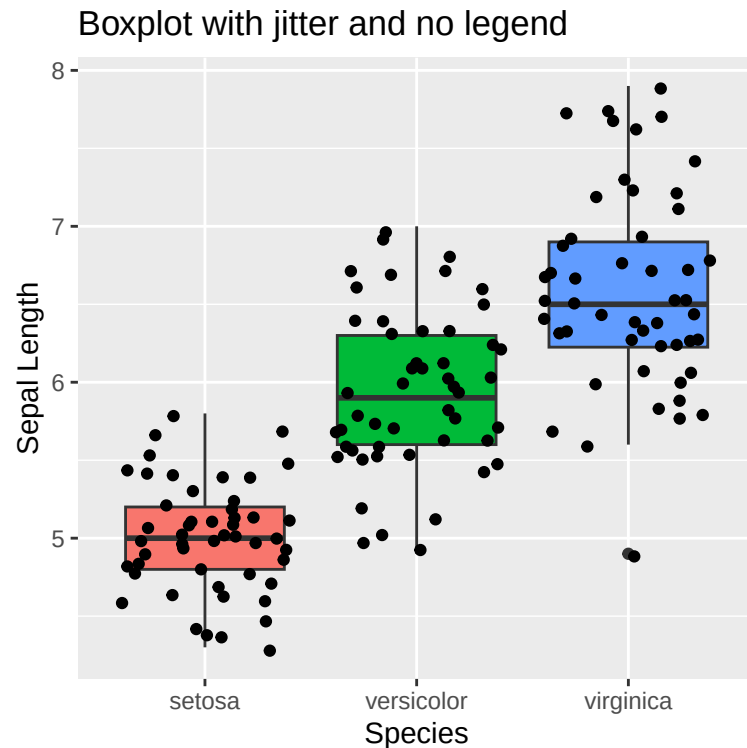
Man kan se, at når man specificerer farver, får man en legende på højre side af plottet. I dette tilfælde er det faktisk ikke nødvendigt, da man kan se uden legende, hvad de tre boxplots refererer til. Derfor fjerner vi legenden fra plottet ved at bruge `theme(legend.position="none")`.

```
ggplot(data=iris, aes(x=Species, y=Sepal.Length, fill=Species)) +
  geom_boxplot() +
  geom_jitter() +
  ylab("Sepal Length") +
  ggtitle("Boxplot with jitter and no legend") +
  theme(legend.position="none")
```



Man kan, som alternativ til ovenstående, fjerne legend ved at tilføje `show.legend = FALSE` som indstilling i den `geom`, der generere den. Være opmærksomme på, at i dette tilfælde skal man skrive `show.legend = FALSE` i både `geom_boxplot()` og `geom_jitter()` funktioner for at få legenden helet fjernet.

```
ggplot(data=iris, aes(x=Species, y=Sepal.Length, fill=Species)) +  
  geom_boxplot(show.legend = FALSE) +  
  geom_jitter(show.legend = FALSE) +  
  ylab("Sepal Length") +  
  ggtitle("Boxplot with jitter and no legend")
```

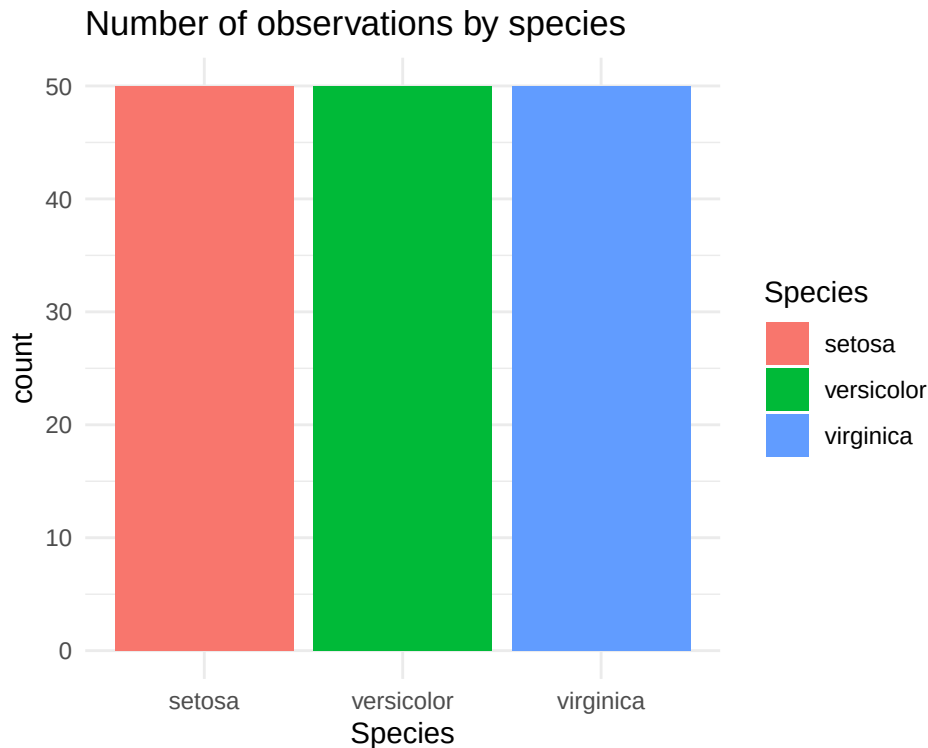


Barplot (geom_bar)

Med `ggplot()` kan man repræsentere data i et barplot ved at bruge `geom_bar()`. Her vil vi gerne tælle antallet af observationer for hver art (variablen `Species`) og visualisere dem som søjler. Indenfor `geom_bar()` specificerer vi derfor `stat="count"`.

Vi bruger også `fill=Species` for at give forskellige farver automatisk for hver af de tre arter. Bemærk, at det var `color=Species` i det forudgående plot, da vi anvendte `geom_point()`. Det skyldes, at `color` bruges til punkter og linjer, mens `fill` er til større områder, der bliver udfyldt, såsom søjler og histogrammer.

```
ggplot(iris, aes(x=Species, fill=Species)) +
  geom_bar(stat = "count") +
  ggtitle("Number of observations by species") +
  theme_minimal()
```



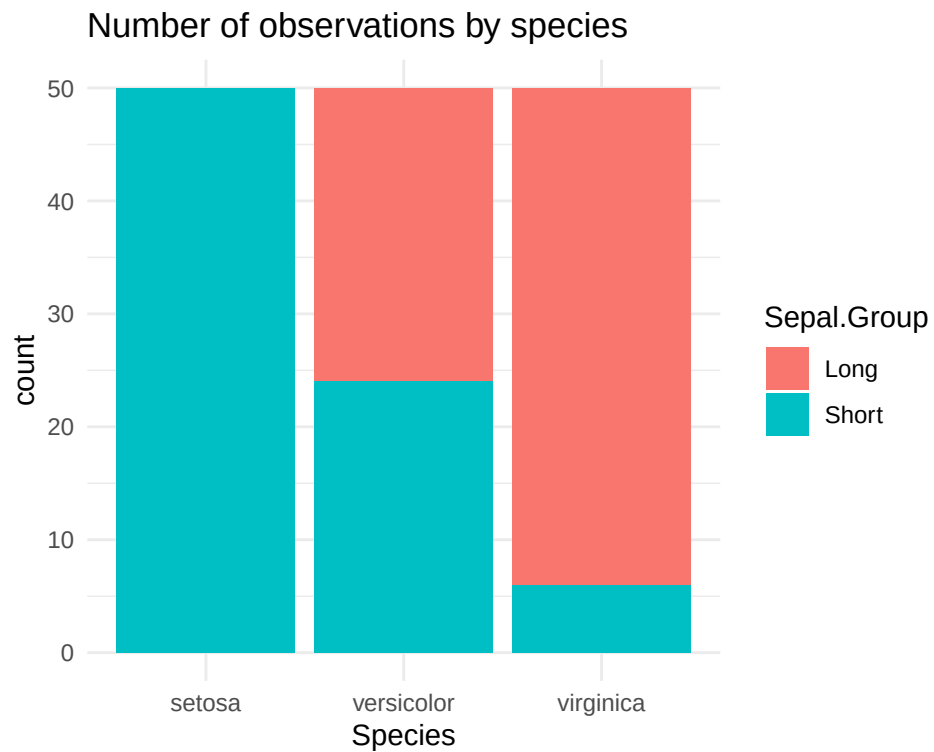
Barplot: stack vs dodge

Hvis man har flere kategoriske variabler, kan man lave barplots på forskellige måder. Til datasættet tilføjer vi en ekstra kategorisk variabel med navn `Sepal.Group`, som adskiller mellem `Long` og `Short` værdier af variabelen `Sepal.Length`. Her specificerer jeg bare (med funktionen `ifelse()`), at hvis `Sepal.Length` er længere end den gennemsnitlige `Sepal.Length`, så er det betragtet `Long`, ellers er det `Short`, som i følgende:

```
iris$Sepal.Group <-
  ifelse(iris$Sepal.Length > mean(iris$Sepal.Length), #test
        "Long",                                     #if TRUE
        "Short")                                    #if FALSE
```

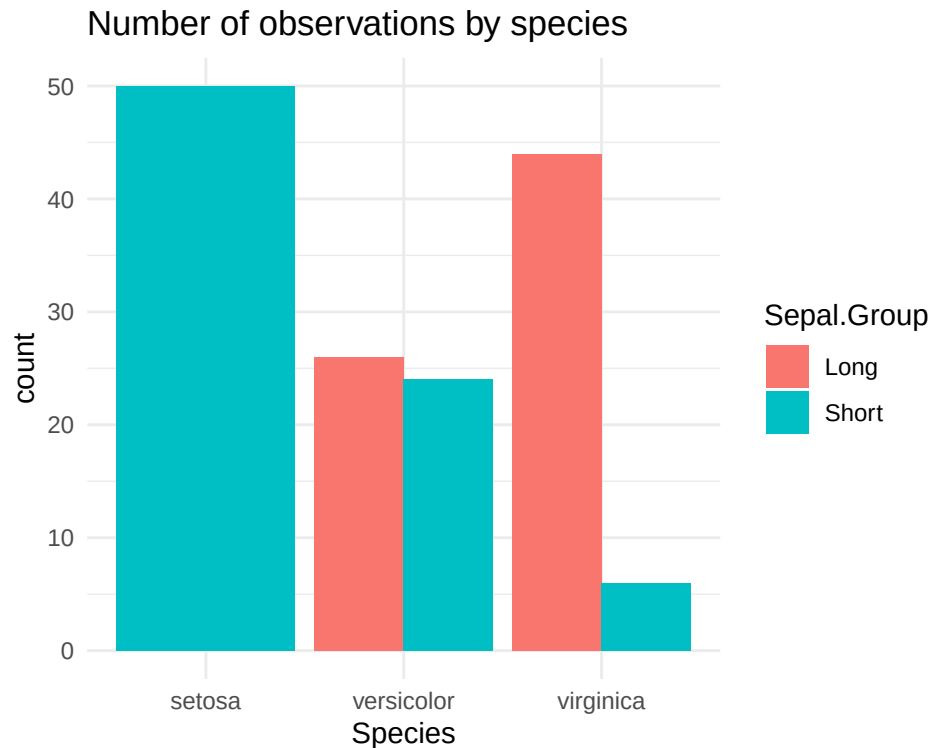
Når vi laver en barplot med de to variabler, tilføjer jeg `Sepal.Group` med `fill`, og `ggplot` splitter antal observationer efter `Sepal.Group` med farver som repræsenterer `Long` eller `Short`, og tilføjer en tilsvarende legende.

```
ggplot(iris, aes(x=Species, fill=Sepal.Group)) +
  geom_bar(stat = "count") +
  ggtitle("Number of observations by species") +
  theme_minimal()
```



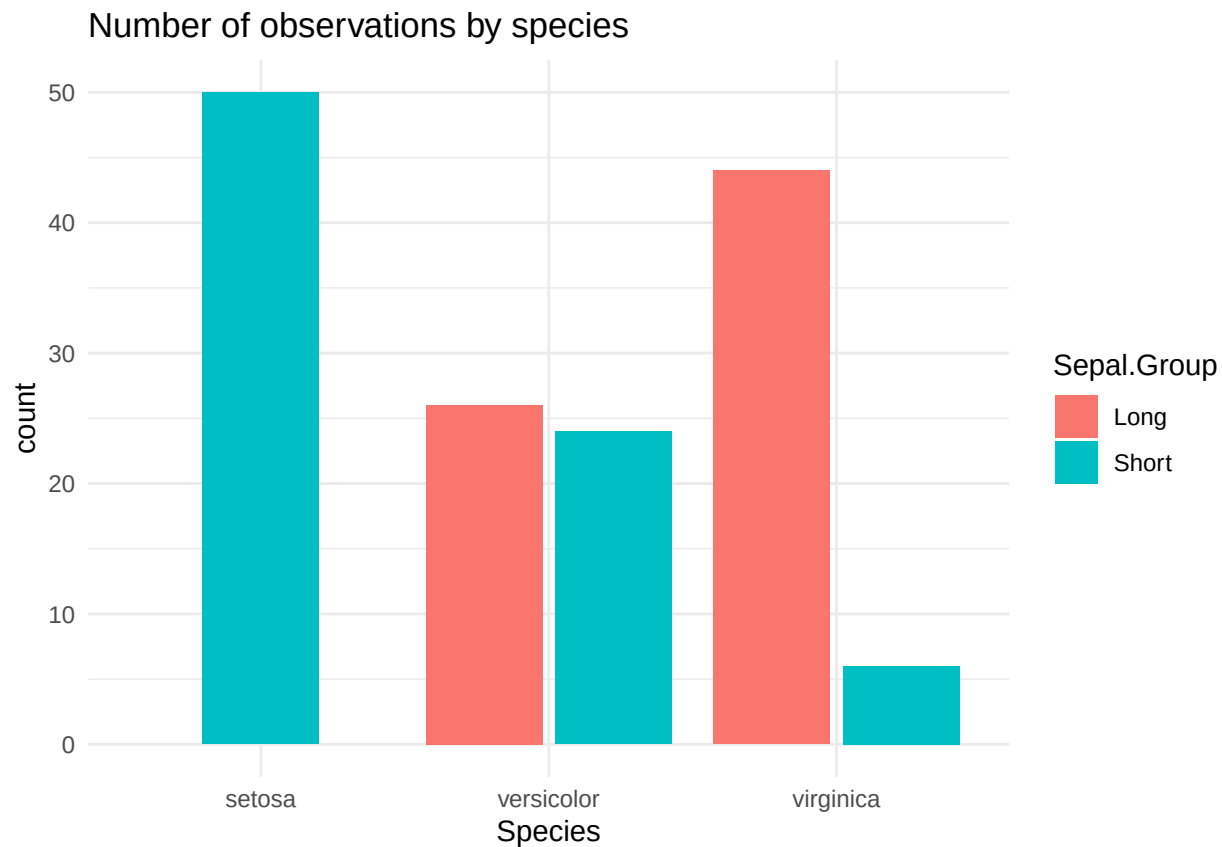
Mange gange foretrækker man at have søjler stående ved siden af i stedet for på toppen af hinanden. Dette kan opnås ved blot at tilføje `position="dodge"` inde i `geom_bar()`.

```
ggplot(iris, aes(x=Species, fill=Sepal.Group)) +  
  geom_bar(stat = "count", position = "dodge") +  
  ggtitle("Number of observations by species") +  
  theme_minimal()
```



Som et eksempel på fleksibiliteten i pakken `ggplot2`, kan jeg nævne, at det kan være irriterende, når bredden af søjlerne for arten `setosa` i et barplot er dobbelt så bred som de andre søjler, fordi der ikke er nogen observationer i `setosa`-gruppen med en `Long` værdi i variabelen `Sepal.Group`. En løsning på dette kan findes ved at tilføje `position=position_dodge2(preserve = "single")` i `geom_bar()`. Denne parameter gør bredden på alle søjler ens, uanset om der er data i alle kategorier eller ej.

```
ggplot(iris, aes(x=Species, fill=Sepal.Group)) +  
  geom_bar(stat = "count", position = position_dodge2(preserve = "single")) +  
  ggtitle("Number of observations by species") +  
  theme_minimal()
```

Histogram (geom_histogram)

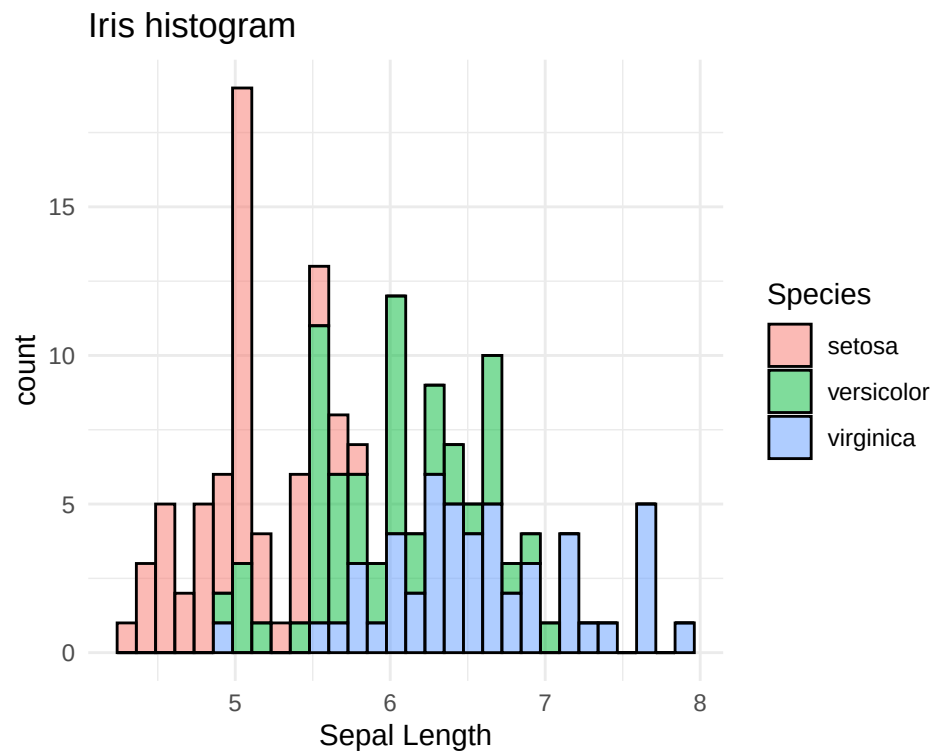
Et histogram bruges til at give et overblik over, hvordan dataene fordeler sig. Med `ggplot2` kan man lave et histogram med `geom_histogram()`. Den x-akse variabel skal være en kontinuerlig variabel. Her specificerer vi, at vi vil gerne have et histogram for hver art (Species).

```
ggplot(data=iris, aes(x=Sepal.Length, fill=Species)) +  
  geom_histogram() +  
  xlab("Sepal Length") +  
  ggtitle("Iris histogram") +  
  theme_minimal()
```



Man kan også gøre det nemmere at skelne mellem de tre arter ved at sætte `alpha=0.5` inden for `geom_histogram` og ved at angive en linjefarve som parameter ind i `geom_histogram()`.

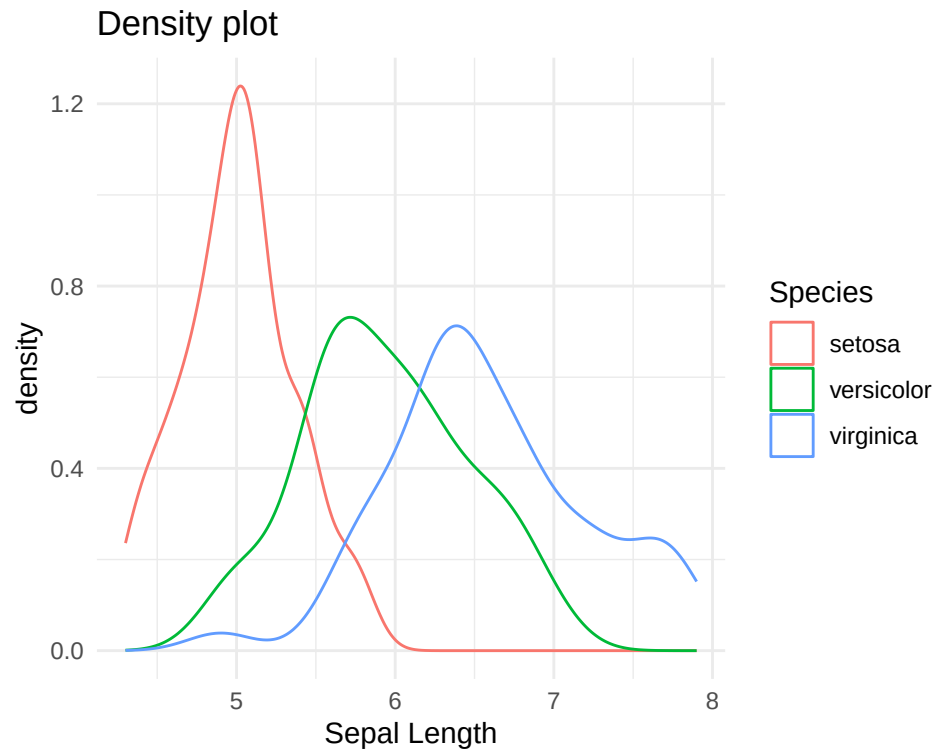
```
ggplot(data=iris, aes(x=Sepal.Length, fill=Species)) +  
  geom_histogram(alpha=0.5,color="black") +  
  xlab("Sepal Length") +  
  ggtitle("Iris histogram") +  
  theme_minimal()
```



Density (geom_density)

Med et density plot kan man, ligesom med et histogram, se fordelingen af dataene i form af en glat eller "smooth" kurve.

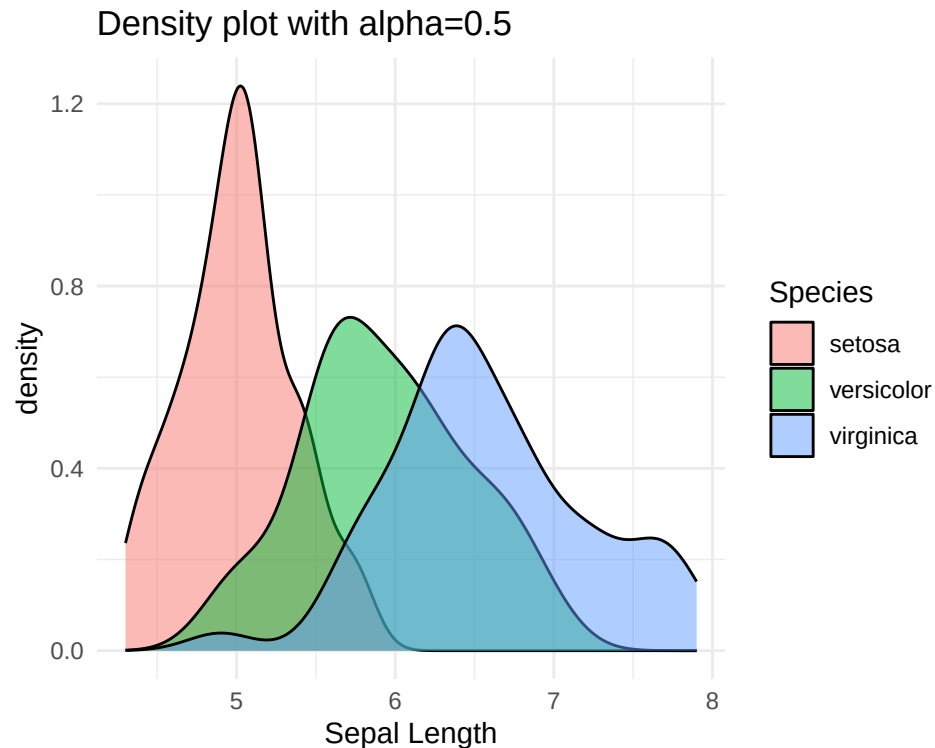
```
ggplot(data=iris, aes(x=Sepal.Length, color=Species)) +  
  geom_density() +  
  xlab("Sepal Length") +  
  ggtitle("Density plot") +  
  theme_minimal()
```



Density plot med fill og gennemsigtig farver

Vi kan angive en værdi for `alpha` indenfor `geom_density()`. Som før, specificerer parameteren `alpha` gennemsigtigheden af density kurverne i plottet.

```
ggplot(data=iris, aes(x=Sepal.Length, fill=Species)) +  
  geom_density(alpha=0.5) +  
  xlab("Sepal Length") +  
  ggtitle("Density plot with alpha=0.5") +  
  theme_minimal()
```

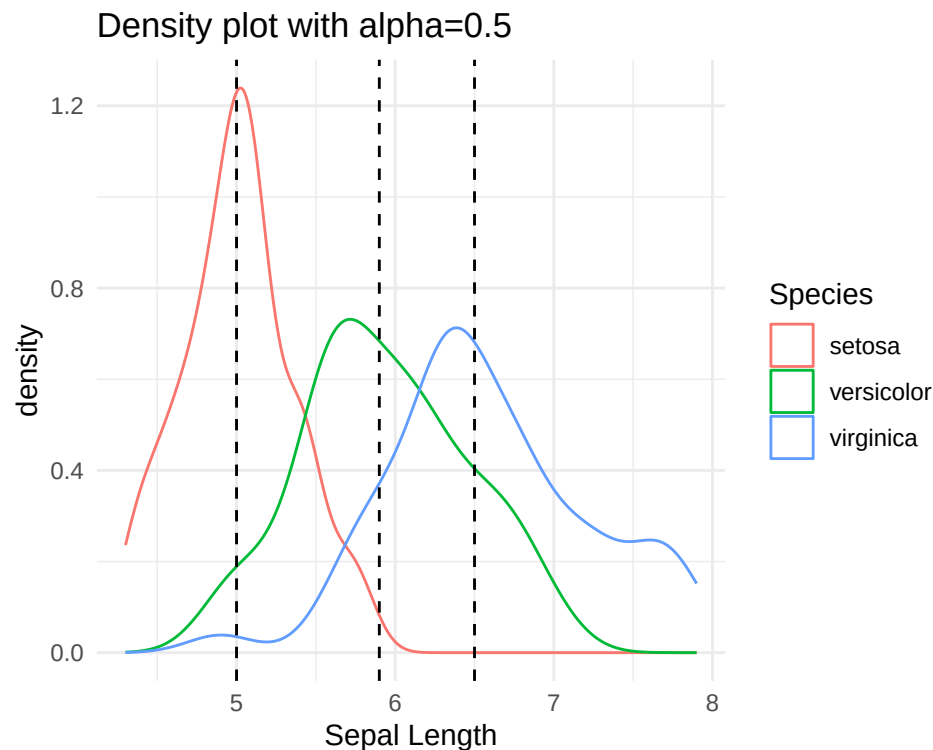


Tilføje middelværdi linjer

Vi bruger funktionen `tapply()` til at beregne middelværdierne af `Sepal.Length` for hver af de tre `Species`. Vi kan derefter tilføje dem som lodrette linjer til vores plot. Her bruger vi `geom_vline()` (OBS, det er `geom_hline()`, hvis man vil have en vandret linje) og fortæller, at `xintercept` skal være lig med de middelværdier, som vi har beregnet. Parameteren `lty=2` (i.e., linetype) betyder, at vi vil gerne have en stiplede ("dashed") linje.

```
means <- tapply(iris$Sepal.Length, iris$Species, median)

ggplot(data=iris, aes(x=Sepal.Length, color=Species)) +
  geom_density(alpha=0.5) +
  xlab("Sepal.Length") +
  ggtitle("Density plot with alpha=0.5") +
  geom_vline(xintercept = means, lty=2) +
  theme_minimal()
```



Line plot (geom_line())

`geom_line()` kan bruges til at lave linjediagrammer. Her indlæser vi datasættet `population` og filtrerer kun landene fra Skandinavien.

```
library(tidyr)
data(population)
population_scand <- population[population$country %in% c("Denmark", "Sweden", "Norway", "Finland"),]
head(population_scand)
```

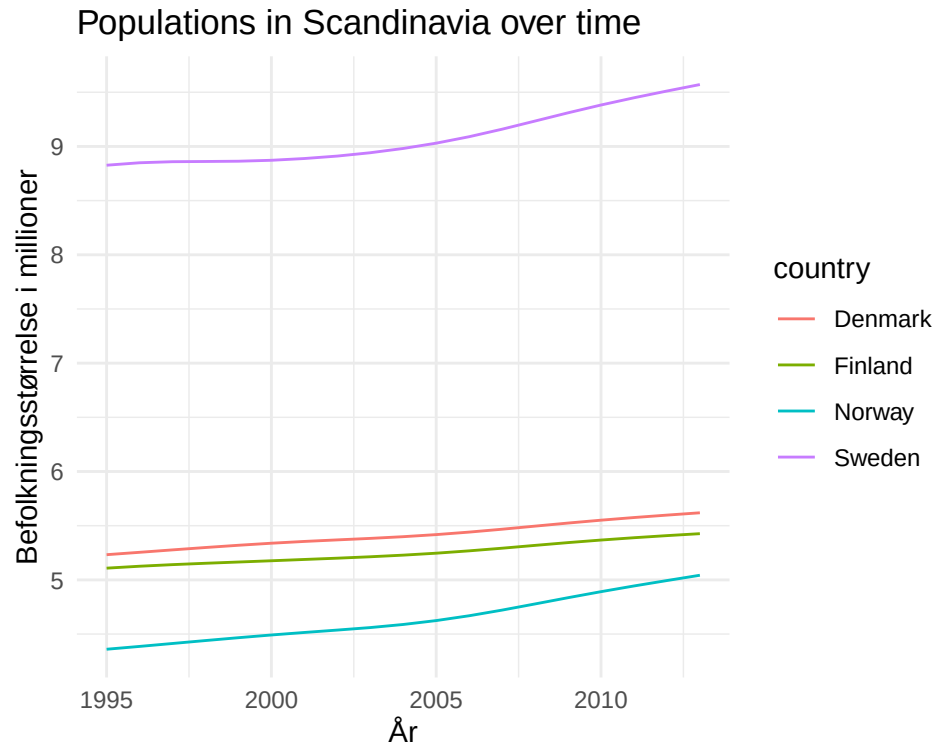
```
## # A tibble: 6 x 3
##   country year population
##   <chr>   <dbl>     <dbl>
## 1 Denmark 1995     5232582
## 2 Denmark 1996     5254383
## 3 Denmark 1997     5276683
## 4 Denmark 1998     5298680
## 5 Denmark 1999     5319410
## 6 Denmark 2000     5338283
```

Når tallene i variablen `population` er ret store, kan det være en god idé at normalisere, så de betegner befolkningsstørrelser i millioner.

```
population_scand$population <- population_scand$population/1e06
```

Her laver vi et plot med `geom_line()`, så vi kan se udviklingen i befolkningsstørrelserne over tid. For at få det til at virke, skal man huske at angive `group` inden for `aes()`, udover at man specificerer farven med `colour`:

```
ggplot(data = population_scand, aes(x = year, y = population, group = country, colour = country)) +
  geom_line() +
  labs(title = "Populations in Scandinavia over time", x = "År", y = "Befolkningsstørrelse i millioner")
  theme_minimal()
```



Troubleshooting

Her er blot en lille liste over nogle aspekter, der kan forårsage fejl, når man kører kode med `ggplot2`. Jeg tilføjer også andre fejl, som kan opstå i vores lektion.

- `ggplot(data=iris, aes(...))`: Husk her, at `data=iris` er korrekt og **ikke** `Data=iris` (R skelner mellem store og små bogstaver). Man kan også fjerne `data=` og skrive bare `iris` i stedet for.
- Forkert stavning: Dobbelttjek, at du har stavet variabel- eller funktionsnavne korrekt.
- Glemte `+`-symbol: For at forbinde komponenterne i plottet skal man huske at tilføje `+` i slutningen af en linje og skrive de næste komponenter bagefter (man behøver ikke at skrive hver komponent på en ny linje, men det gør det nemmere at læse koden).
- Skrev `%>%`-symbolet i stedet for `+`: De øvrige pakker fra **tidyverse** bruger `%>%`.
- Glemte parentes: Her har man glemte den sidste parentes, der skal være `fill=Species))` og **ikke** `fill=Species)`. Man får bare en `+` i consol vinduet, fordi R forventer, at man fortsætter med at skrive mere kode.

```
ggplot(data=iris, aes(x=Sepal.Length, fill=Species))
+
```

- `fill` og `colour`: indenfor `aes()` refererer `fill` til at man udfylder f.eks. bars eller regioner med farver, og `colour` refererer til farven af linjer eller punkter.

Problemstillinger

1) Quiz på Absalon - den hedder Quiz - ggplot2 part 1.

OBS! Husk at lave følgende øvelser i R Markdown. Det er god praksis at sikre, at jeres dokument kan knytte - i selve eksamen afleverer du et html dokument.

- *Lav et nyt R Markdown-dokument og fjern eksempelkode. Husk at oprette en ny chunk ved at trykke på "Insert a new chunk" eller bruge genvejstastene CMD+ALT+I eller CTRL+ALT+I. Jeg anbefaler at oprette en ny chunk for hver plot, I laver.*

Vi bruger datasættet `diamonds`. Husk at først indlæse dataene:

```
data(diamonds)
```

Her er beskrivelsen af `diamonds`:

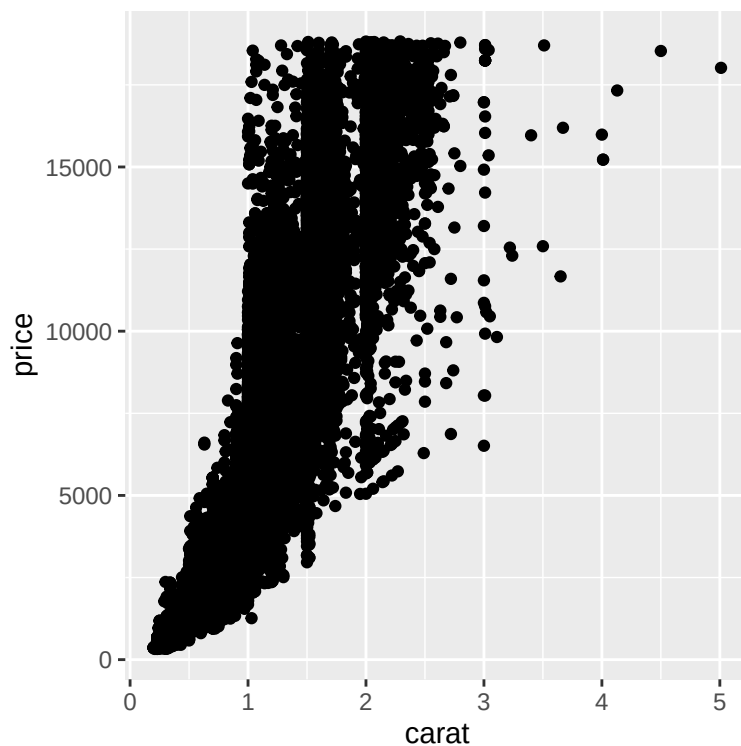
Prices of over 50,000 round cut diamonds: a dataset containing the prices and other attributes of almost 54,000 diamonds.

Se også `?diamonds` for en beskrivelse af variablerne.

2) Brug datasættet `diamonds` til at lave et spredningsdiagram / scatter plot (`geom_point()`) med:

- `carat` på x-aksen
- `price` på y-aksen

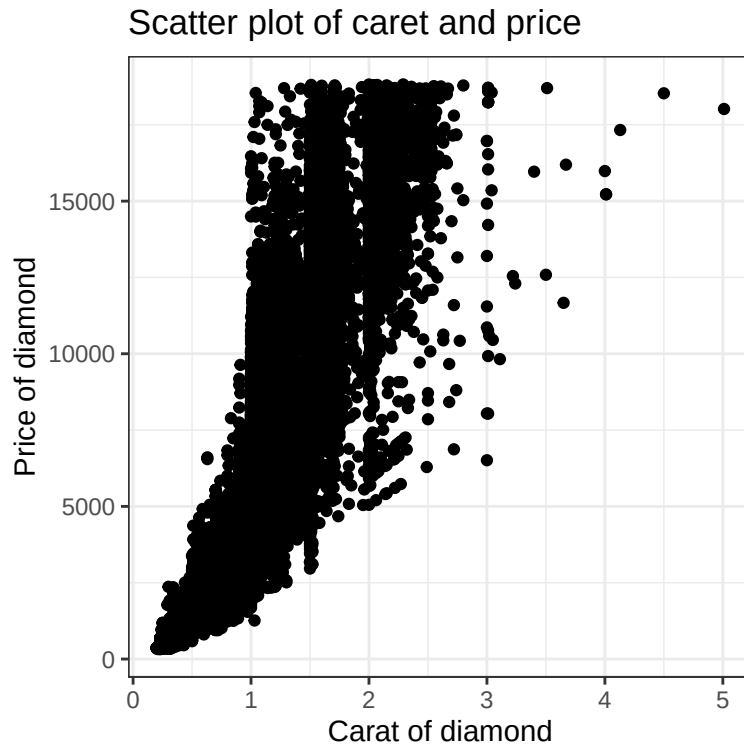
Som reference og til sammenligning: Dit plot skal se sådan ud:



3) Tilføj følgende komponenter til dit plot fra 2):

- En x-akse etiket / label (`xlab()`) og en y-akse etiket / label (`ylab()`)
- En titel (`ggtitle()`)
- Et tema som hedder `theme_bw()`
- Husk at forbinde komponenterne med `+` og skriv de nye komponenter hver på en ny linje.

Det skal se sådan ud:



4) Ændr temaet på dit plot til `theme_classic()` eller `theme_minimal()` i stedet for `theme_bw()` og se på resultatet.

- Hvis man (måske ved et uheld) skriver **to** temaer på samme tid (f.eks. `+ theme_bw() + theme_classic()`), hvilket tema vil så blive anvendt i plottet?
- Valgfri ekstra: her er nogle flere temaer, du kan prøve: <https://ggplot2.tidyverse.org/reference/ggtheme.html>.

5) Lav det samme plot som i 3), og skriv `color=color` ind i `aes()`. Den første `color` refererer til punkt farver og den anden til variablen `color` i dataframen.

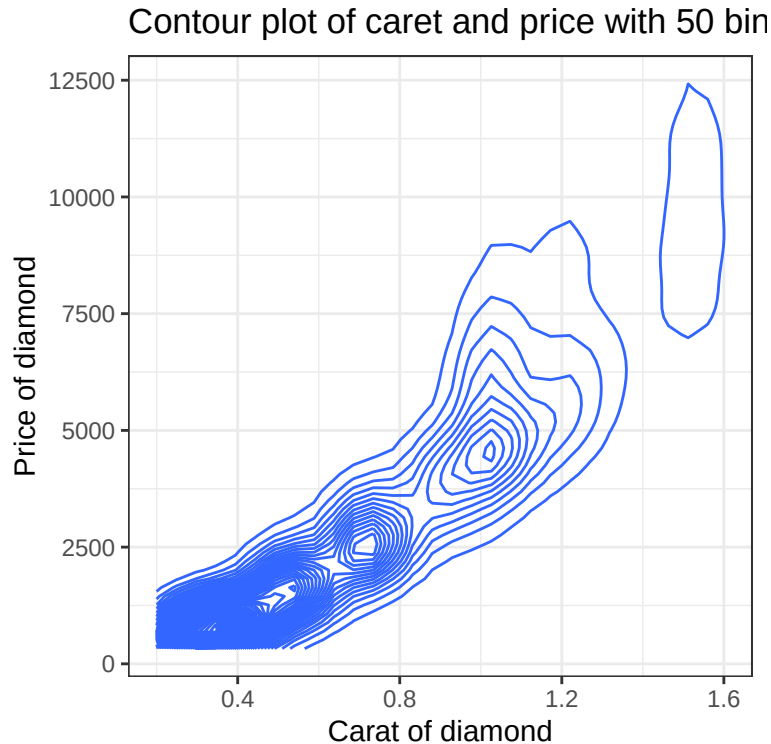
- Nu fjern `color=color` fra funktionen `aes()` og i stedet tilføj `aes(color=color)` i funktionen `geom_point()`. Får du så samme resultat?
- Bemærk at det er det samme om man bruger britisk eller amerikansk stavning i `ggplot2` - f.eks. `colour` eller `color` inde i `aes()` giver samme resultat.

6) Bemærk at `diamonds` er en ret stort datasæt. Tjek, hvor mange observationer datasættet har, og prøv derefter følgende:

- Anvend `sample()`-funktionen til at udtage kun 1000 tilfældige rækker fra datasættet og lav dit punktplot igen. (OBS! Se venligst kapitel 1 hvis du er usikker med `sample()`.)

- En mulig løsning, der bruger hele datasættet men uden at have mange punkter på plottet, er at lave en contour plot ved at erstatte `geom_point()` med `geom_density_2d()`. Prøv det.

Det ser sådan ud: (OBS! Vi bruger indstilling `bins=50` ind i `geom_density_2d()` men prøv venligst at justere på antallet af bins).

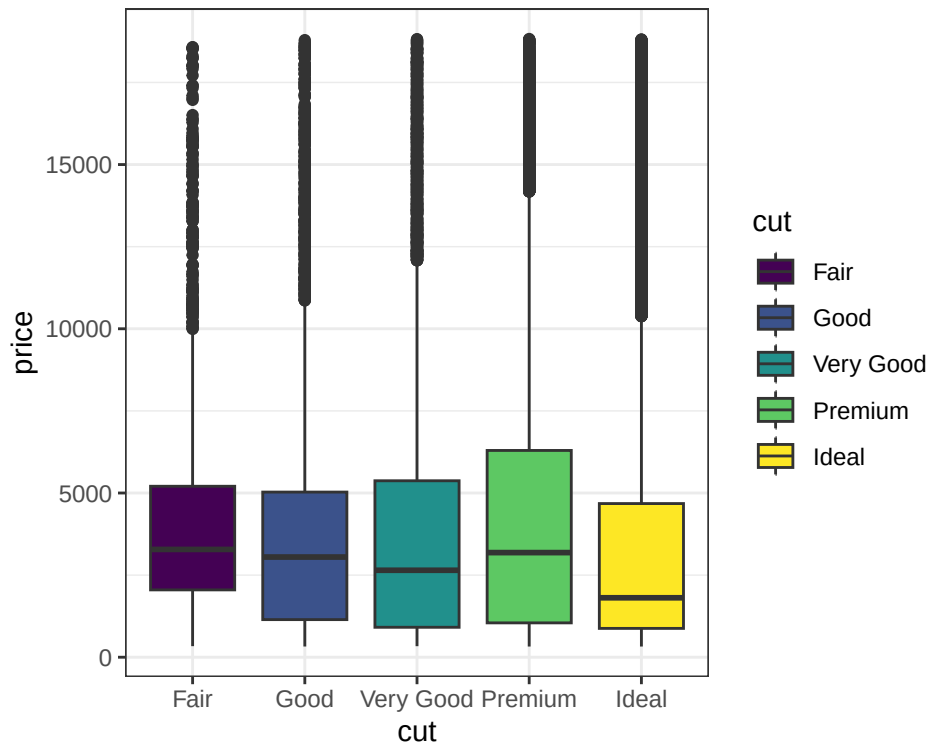


Som ekstra kan du prøve indstillingen `geom = "polygon"` inde i `stat_density_2d`. Der er andre muligheder her.

7) Fortsæt med at anvende `diamonds` datasættet til at lave et boxplot med:

- `cut` på x-aksen (giv x-aksen label `Cut`).
- `price` på y-aksen (giv y-aksen label `Price of diamond`).
- Brug `fill` til at give forskellige farver til de mulige værdier af `cut`.
- Brug temaet `theme_bw()`.

Det skal se sådan ud:

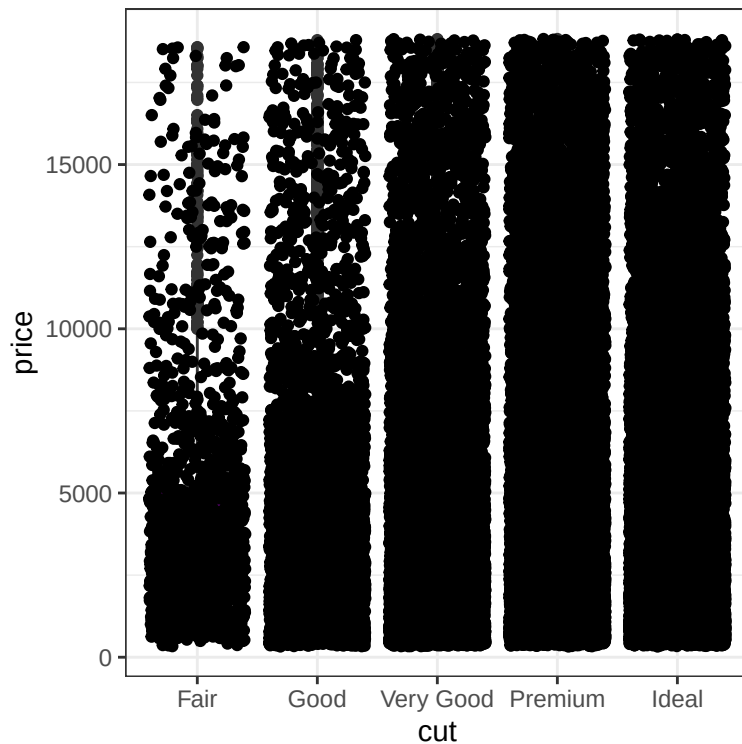


- Hvordan ser det ud, hvis man bruger `colour` i stedet for `fill`? Eller hvis man specificerer begge to?

8) Lav følgende ekstra ændringer til din boxplot fra ovenstående:

- Tilføj `geom_jitter()` til din boxplot.
- Fjern legend ved at tilføj `theme(legend.position="none")`.
- Man kan også tilføj `show.legend=FALSE` til både `geom_boxplot()` og `geom_jitter()` i stedet - prøv det i stedet for `theme(legend.position="none")`. Er det nok at tilføje `show.legend=FALSE` til kun én af de to geoms?

Dit resultat skal se sådan ud:



- Man kan også prøve at forbedre plottet ved at give nogle indstillinger ind i `geom_jitter()`, for eksempel kan man prøve `geom_jitter(size=.2,color="grey",alpha=0.5)` så at punkterne fylder mindre i plottet (ellers kan man overveje at fjerne dem).

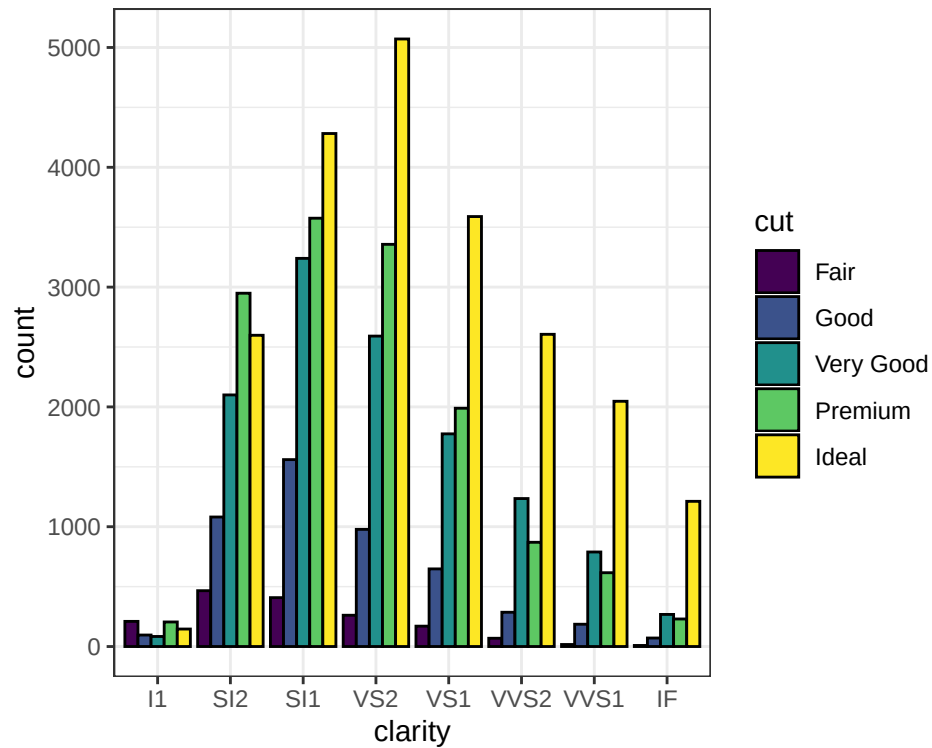
Leg med de tre indstillinger `size`, `color` og `alpha` og se på forskellen. Her er mere detaljer vedrørende `alpha`:

Alpha refers to the opacity of a geom. Values of alpha range from 0 to 1, with lower values corresponding to more transparent colors. https://ggplot2.tidyverse.org/reference/aes_colour_fill_alpha.html

- Prøv at skifte rækkefølgerne af `geom_jitter()` og `geom_boxplot()` i dit plot kommando og se - gøre det en forskel til, hvordan ser plottet ud?

9) Lav en barplot med indstillingen `stat="count"` og:

- Variablen `clarity` på x-aksen.
- Forskellige farver til gruppevariablen `cut`.
- Specificer `position="dodge"` for at få bars ved siden af hinanden.
- Brug også indstillingen `color="black"` og noter effekten.
- Tilføj et tema.

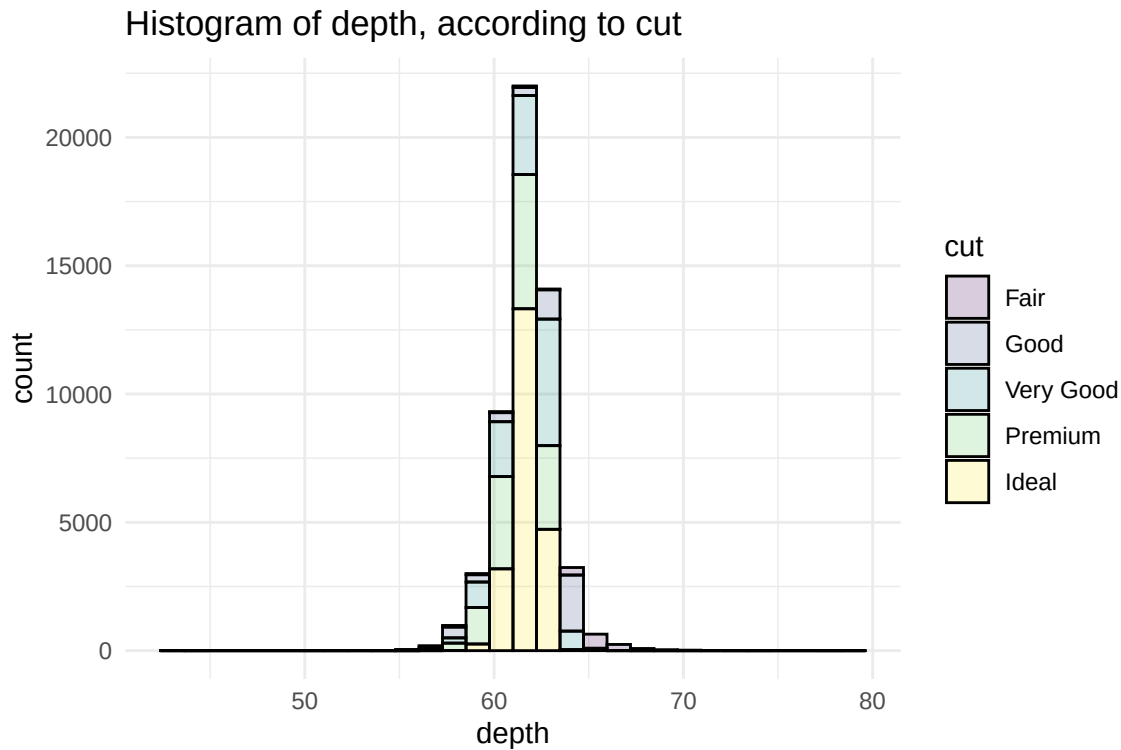


10) Lav en histogram med:

- Variablen `depth` på x-aksen.
- Forskellige farver til gruppevariablen `cut`.
- Brug indstilling `alpha` til at ændre gennemsigtigheden af kolonnerne.
- Giv kolonnerne en sort ramme.
- Tilføj et tema osv.

Dit resultat skal se sådan ud:

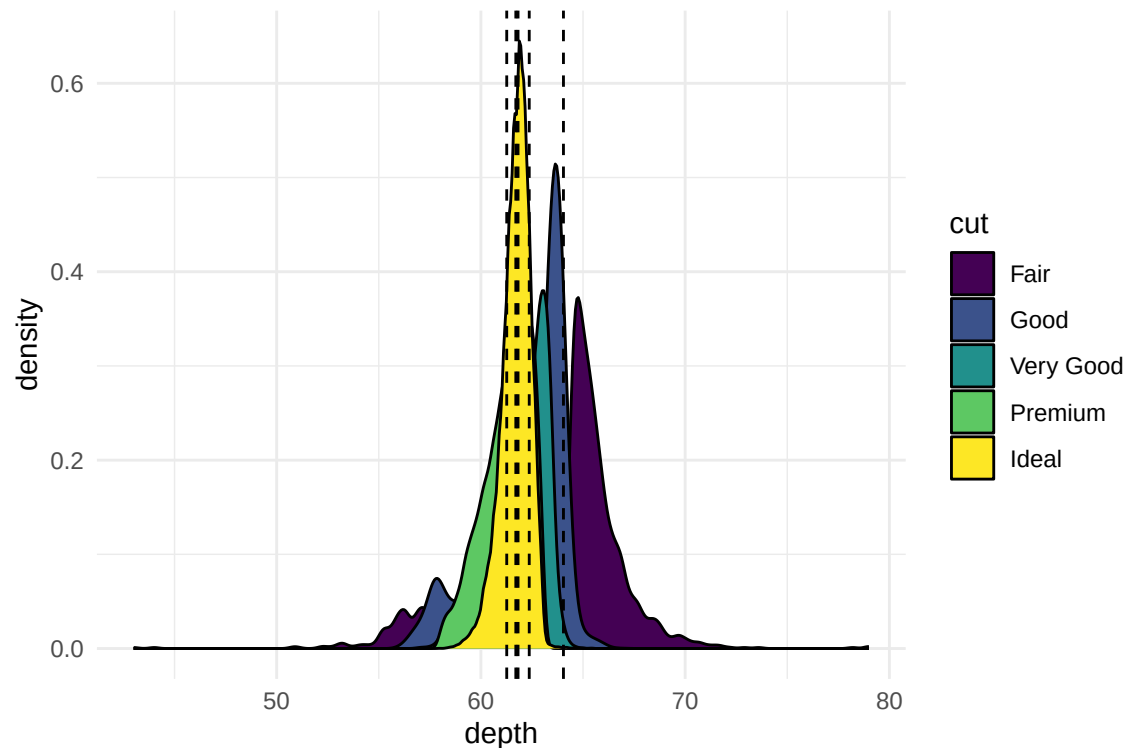
```
## 'stat_bin()' using 'bins = 30'. Pick better value 'binwidth'.
```



- Tip: Ændr på parameteren `bins` ind i `geom_histogram()`-funktionen.

11) Lav et density plot:

- Det kan være svært at sammenligne fordelingerne i de histogrammer som du har oprettet tidligere.
- Erstat `geom_histogram` med `geom_density` i din kode fra 9).
- Er det nu lettere at sammenligne fordelingerne efter de forskellige niveauer af `cut`?
- Tilføj lodrette linjer med beregnede medianværdier af variabelen `depth` for hver af de `cut`-niveauer.
 - Tip: Brug `tapply` til at beregne medianværdierne og `geom_vline` til at tilføje lodrette linjer.



- Ekstra: Hvis du vil have linjerne til at passe sammen med farverne i dine density kurver, så se venligst den næste code chunk og prøv at forstå koden.

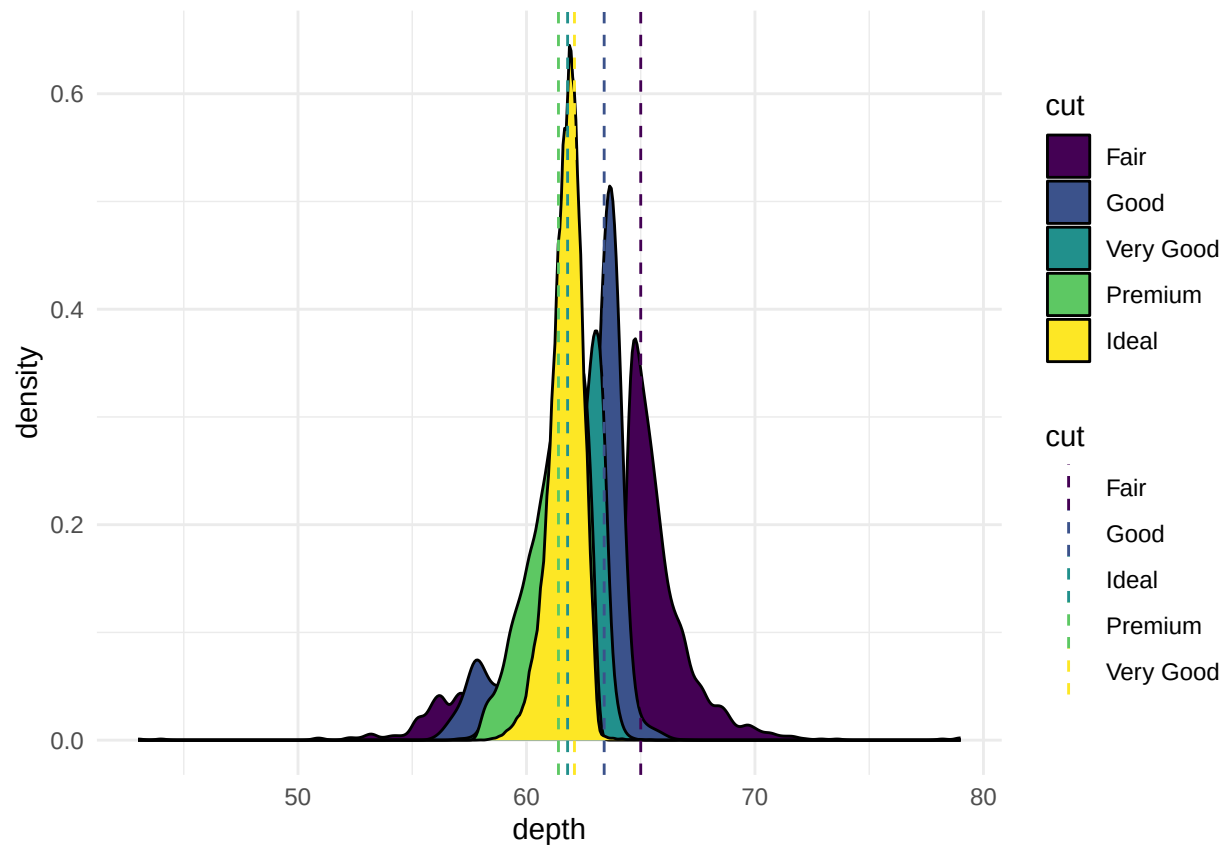
```
#calculate medians
mymedians <- ## din tapply funktion her

#make a data frame containing the means and cut (as an ordinal variable to match the dataset itself - s
median_df <- data.frame("mean" = mymedians, "cut" = as.ordered(names(mymedians)))

#use the data frame in geom_vline and use local aes() in plot

#### TODO: ERSTAT geom_vline() med følgende:
#### geom_vline(data = median_df, aes(xintercept = mean, colour=cut),lty=2)
```

Det endelige resultat:



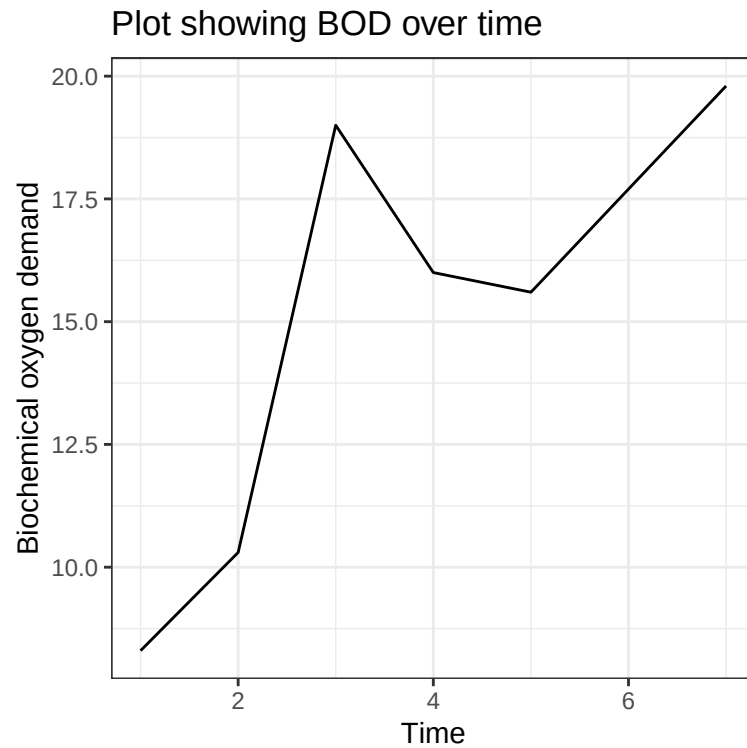
12) Lav et simple line plot.

Åbn datsættet BOD:

```
data(BOD)
```

#BOD: This dataset contains the biochemical oxygen demand (BOD) of water samples taken at different times

- Plot variablen `Time` på x-aksen.
- Plot variablen `demand` på y-aksen.
- Anvend `geom_line()`.
- Vælg selv etiketter / labels, title og tema.



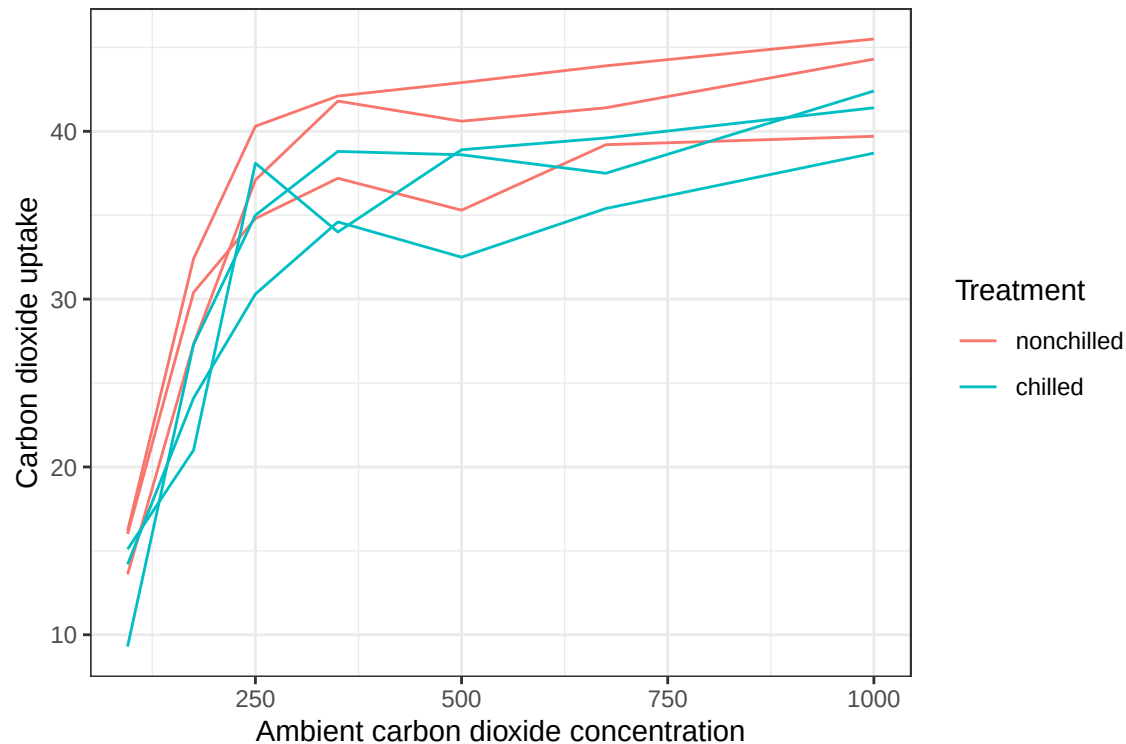
13 Lav et line plot (`geom_line()`).

Åbn datasættet `C02`:

```
data(C02)
```

- Lav en delmængde til typen “Quebec” (variablen `Type`).
- Derefter, anvend din delmængde til at lave et line plot med variablen `conc` på x-aksen, variablen `uptake` på y-aksen.
- Dit plot burde have seks linjer (en til hver plant i variablen `Plant`).
- Linjerne farves efter behandlingsmetode (variablen `Treatment`).

Det endelige resultat kunne se sådan ud:



14) En ekstra øvelse: Arbejd frit med at lave andre plots fra nogle af de andre indbygget datasæt i R (indtast `datasets::` i konsolen for at fremkalde en liste over muligheder).

I alle tilfælde, tilføj akse-labels, en titel, et tema osv.

Næste gang

Efter at have lavet disse øvelser vil man kunne se, at der er rigtig meget fleksibilitet involveret i at lave et plot med `ggplot2`. I den næste lektion kommer vi til at fortsætte med andre plot-typer og lære, hvordan man f.eks. manuelt kan vælge farver.